$J \int Skripte$

Kryptologie ICS.KRYPTOG

Folien zu den Präsenz 2 & 3
„Symmetrische Kryptographie“, HS 26

Die Folien 5 – 8 (HEX & XOR) sind auf die Präsenz 2 als HA vorzubereiten (flipped classroom)

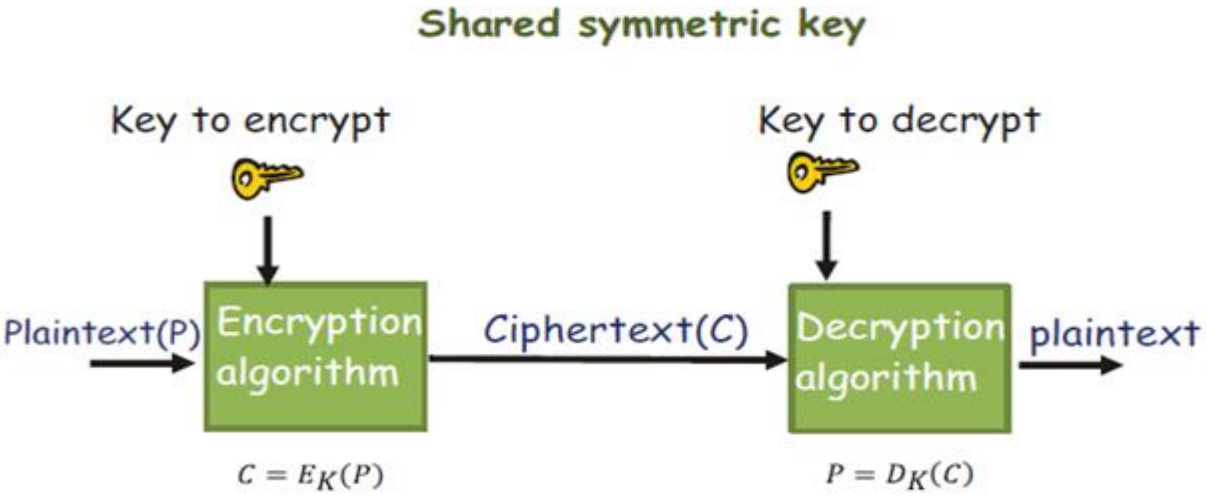
Link des Tutorials:

https://hslu.zoom.us/rec/share/5Z_7xAIyBUNCxVMjT2mbf4iBE-PFMijvxAv8JPtb9AKLGFvhSdpx590St75UTivR.525CbCNu3GNkBHAN

Die Folien 23 – 34 (Fakultät & Bin.koeff.) sind auf die Präsenz 3 als HA vorzubereiten (flipped classroom)

Link des Tutorials:

https://hslu.zoom.us/rec/share/1SDSdZ0UDgod3jXrSaSq2lK9i3VFJftZv9L6VWdh9M2cmamh7xE--y2XTT2nIXTK.LLmz-xlxTq1m-Dhk



Graphik zur symmetrischen Verschlüsselung aus „New Frontiers in Cryptography“, Khaled Salah Mohamed, Springer Verlag

©Josef Schuler, dipl. math., dipl. Ing. NDS ETHZ, MSc Applied IT-Security, Feldhof 25, 6300 Zug, j.schuler@bluewin.ch resp. josef.schuler@hslu.ch

Inhaltsübersicht

- Eine Einführung in die symmetrische Kryptographie
  - Diverse Funktionsprinzipien (Stromchiffre versus Blockchiffre) = Kap. 7.1 & 7.2
  - Diverse Funktionstechniken und –Modi von Blockchiffren = Kap. 8
- Mathematische Grundlagen: Bitte bereiten Sie die Folien 5 – 8 & 23 – 34 vor.
  - XOR-Operation (von HEX-Zeichen) = Folien 5 – 8; bitte auf Präsenz 2 vorbereiten.
  - Fakultät = Folien 23 – 28; bitte auf Präsenz 3 vorbereiten.
  - Binomialkoeffizienten = Folien 29 – 34; bitte auf Präsenz 3 vorbereiten.

Lernziele

- Ich habe ein gutes Verständnis der Stromchiffren und Blockchiffren erlangt.
- Ich kenne die verschiedenen Modi der Blockchiffren und kann deren Stärken und Schwächen benennen.
- Ich kann eine krypt. Zeichnung in math. Form überführen und umgekehrt.
- Ich kenne den Begriff der perfekten Sicherheit.
- Ich habe die Zusammenfassung zur Vorbereitung (= Präsenz 0) und den Inhalt der ersten drei Präsenzen (= Präsenz 1 – 3) bearbeitet (z.B. die Basisaufgaben und die weiteren Aufgaben in den Folien und im Skript gelöst) und die – erweiterten – Grundlagen der Kryptographie verstanden.

Die Slogans zu dieser Präsenz

Zu Stromchiffren SC: Eine SC bietet keine Integrität!

Zu Blockchiffren BC: BC sind das „work horse“ der Sicherheit!

©Josef Schuler J∫Skripte

Verweise zur Literatur

- ***Vorb. math. Grundlagen (HEX, XOR, Fakultät, & Binomialkoeffizient):***
  - JS Skript „Einführung in die Kryptologie“, Kap. 19.3 & 19.5.
- ***Für die Theorie:***
  - JS Skript „Einführung in die Kryptologie“, Kap. 7.1, 7.2 & 8. Wobei das Kap. 7.2.9 Nachtrag: Feistel-Chiffre ***nicht Prüfungsstoff*** ist.
  - JS Skript „Einf. in die Krypt.“, Kap. 28 „Perfekte Sicherheit“ (Begriff kennen, die Details und Aufgaben sind ***nicht Prüfungsstoff***).
  - In [CP-D] die Kap. 2.1, 2.2 zu Stromchiffren und Kap. 5 zu Blockchiffren.
- ***Aufgaben***
  - Aufgaben in den Folien & JS Skript „Einf. in die Kryptologie“, Kap. 7.1, 7.2 & 8.
  - JS Skript „Aufgaben und Lösungen zum Modul KRYPTOG an der HSLU-I“, Kap. 2.1, „Aufgaben zu den Präsenzen 1 – 3, Grundlagen & sym. Kryptographie“.
- ***Bemerkungen:***
  - Die Kapitelnummerierung in den folgenden Folien entspricht derjenigen im oben erwähnten JS Skript „Einführung in die Kryptologie“.
  - ***Die Aufgaben- und Beispielnummerierung im JS Skript „Einführung in die Kryptologie“ und in den vorliegenden Folien stimmen nicht überein!***
  - ***Die Quellenangaben sind im JS Skript, nicht aber in den Folien enthalten!***
- ***Wichtig:*** Die (#) bezeichn. Folien, Teile & Aufgaben sind ***nicht*** prüfungsrelevant!!

©Josef Schuler $J\int Skripte$

Kap. 7
FUNKTIONSPRINZIPIEN VON ALGORITHMEN

REPETITION AUS DMATH
KAP. 19.3 DIE XOR-OPERATION
= TEIL DER VORBEREITUNG (FLIPPED CLASSROOM)

Die binäre XOR-Operation

Die „XOR“-Verknüpfung heisst auch: exclusive OR, entweder oder, Addition mod 2

p	q	p ⊕ q
1	1	0
1	0	1
0	1	1
0	0	0

Der Ausgang ist 0 genau dann wenn beide Eingänge gleich sind. Resp. der Ausgang ist 1 genau dann wenn beide Eingänge unterschiedlich sind.

in Worten: $\oplus$ = „XOR“

Regel: $p \oplus q = q \oplus p$ (KG)

$(p \oplus q) \oplus r = p \oplus (q \oplus r)$ (AG)

Bemerkungen:

1) In der Kryptographie u.a. bei den sogenannten Stromchiffrierer ist das XOR die zentrale Operation.
2) Im Deutschen entspricht das XOR dem „entweder oder“. D.h. man muss sich für eines der beiden entscheiden, um „wahr“ zu erhalten.

Die XOR-Operation mit HEX Zeichen

In der angewandten Kryptographie arbeitet man üblicherweise mit Halbbytes.
M.a.W. Meldungen, Schlüssel usw. werden in HEX dargestellt.

Tabelle:

HEX		Bit		HEX		Bit
0	=	0000		**8**	=	1000
1	=	0001		**9**	=	1001
2	=	0010		**A**	=	1010
3	=	0011		**B**	=	1011
4	=	0100		**C**	=	1100
5	=	0101		**D**	=	1101
6	=	0110		**E**	=	1110
7	=	0111		**F**	=	1111

Beispiel 1: Wir berechnen nun 7 F ⊕ A B.

1. Übersetzung der Halbbytes in die Bitdarstellung: 0111 1111 ⊕ 1010 1011
2. Addition mod 2 Bit pro Bit (beachten Sie die Farben): 1101 0100
3. Übersetzung der Bitdarstellung wieder in HEX: D4
4. Oder als Alternative mit der XOR-Tabelle cf. nächste Folie

©Josef Schuler *J Skripte*

Die XOR-Operation mit HEX Zeichen

⊕	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
1	1	0	3	2	5	4	7	6	9	8	B	A	D	C	F	E
2	2	3	0	1	6	7	4	5	A	B	8	9	E	F	C	D
3	3	2	1	0	7	6	5	4	B	A	9	8	F	E	D	C
4	4	5	6	7	0	1	2	3	C	D	E	F	8	9	A	B
5	5	4	7	6	1	0	3	2	D	C	F	E	9	8	B	A
6	6	7	4	5	2	3	0	1	E	F	C	D	A	B	8	9
7	7	6	5	4	3	2	1	0	F	E	D	C	B	A	9	8
8	8	9	A	B	C	D	E	F	0	1	2	3	4	5	6	7
9	9	8	B	A	D	C	F	E	1	0	3	2	5	4	7	6
A	A	B	8	9	E	F	C	D	2	3	0	1	6	7	4	5
B	B	A	9	8	F	E	D	C	3	2	1	0	7	6	5	4
C	C	D	E	F	8	9	A	B	4	5	6	7	0	1	2	3
D	D	C	F	E	9	8	B	A	5	4	7	6	1	0	3	2
E	E	F	C	D	A	B	8	9	6	7	4	5	2	3	0	1
F	F	E	D	C	B	A	9	8	7	6	5	4	3	2	1	0

Fortsetzung Beispiel 1: **7 F ⊕ A B = D 4**

Der Schnittpunkt von Zeile und Spalte ist das Resultat; da 7 ⊕ A = A ⊕ 7 = D ist, ist die Reihenfolge unerheblich!

Aufgabe 19.1 Berechnen Sie nun 6 B ⊕ A 5 mit beiden Varianten (d.h. übersetzen in Bit, resp. mit der XOR Tab.)

Aufgabe 19.2 Was für ein Resultat gibt es, falls man zwei gleiche Ausdrücke miteinander „XORed“?

©Josef Schuler *J Skripte*

Einschub Kap. 28
ANHANG 6
PERFEKTE SICHERHEIT (*)

(*) Nur der Inhalt der Folien ist prüfungsrelevant!

Perfekte = informationstheoretische Sicherheit

- … heisst kurz gesagt, dass man – selbst mit unendlich vielen Ressourcen – nicht besser ist als mit raten.
- … heisst, dass ein Chiffrat keine Informationen über den Klartext liefert. Das kann mathematisch wie folgt beschrieben, resp. definiert werden:

  Es sei $\mathcal{M}$ die Menge aller möglichen Klartexte und C die Menge aller möglichen Chiffrate. Perfekte Sicherheit gilt, wenn $\forall\, m \in \mathcal{M}$ und $\forall\, c \in C$ gilt, dass

  $$p(m|c) = p(m).$$

- … heisst schlussendlich, dass die Wsk. den Klartext aus dem gegebenen Chiffrat zu bestimmen gleich ist, wie das reine Erraten des Klartextes, cf. Gl. $p(m|c) = p(m)$.
- … liefern insbesondere ganz einfache Operationen wie XOR, Addition mod N, Multiplikation mod p usw.
- … hat als Vorbedingung, dass der gewählte Schlüssel absolut zufällig ist.
- … liefern insbesondere ganz einfache Operationen wie XOR, Addition mod N, Multiplikation mod p usw.

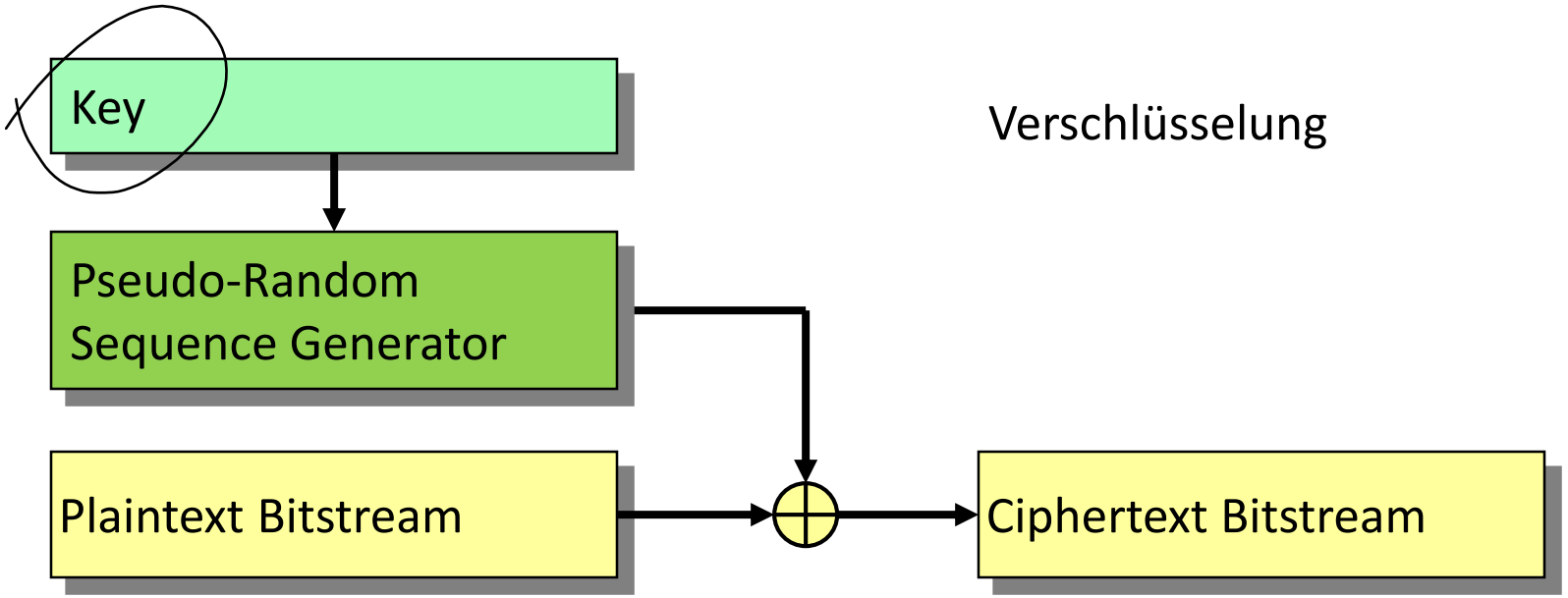
Beispiel 2:

Betrachten wir die folgende Situation. Wir verschlüsseln jeweils nur ***einen*** Buchstaben mit der Caesar-Chiffre und wählen jedes Mal den Schlüssel $k \in K = \mathbb{Z}_{26}$ zufällig und mit gleicher Wahrscheinlichkeit.
Dann ist diese Verschlüsselung ***eines*** Buchstabens perfekt sicher, d.h. selbst mit unendlich vielen Ressourcen ist man mit allen möglichen Analysen nicht erfolgreicher als mit raten.

Aufgabe 28.1 Da muss es ja wohl einen Haken geben, oder? Klären Sie auf!

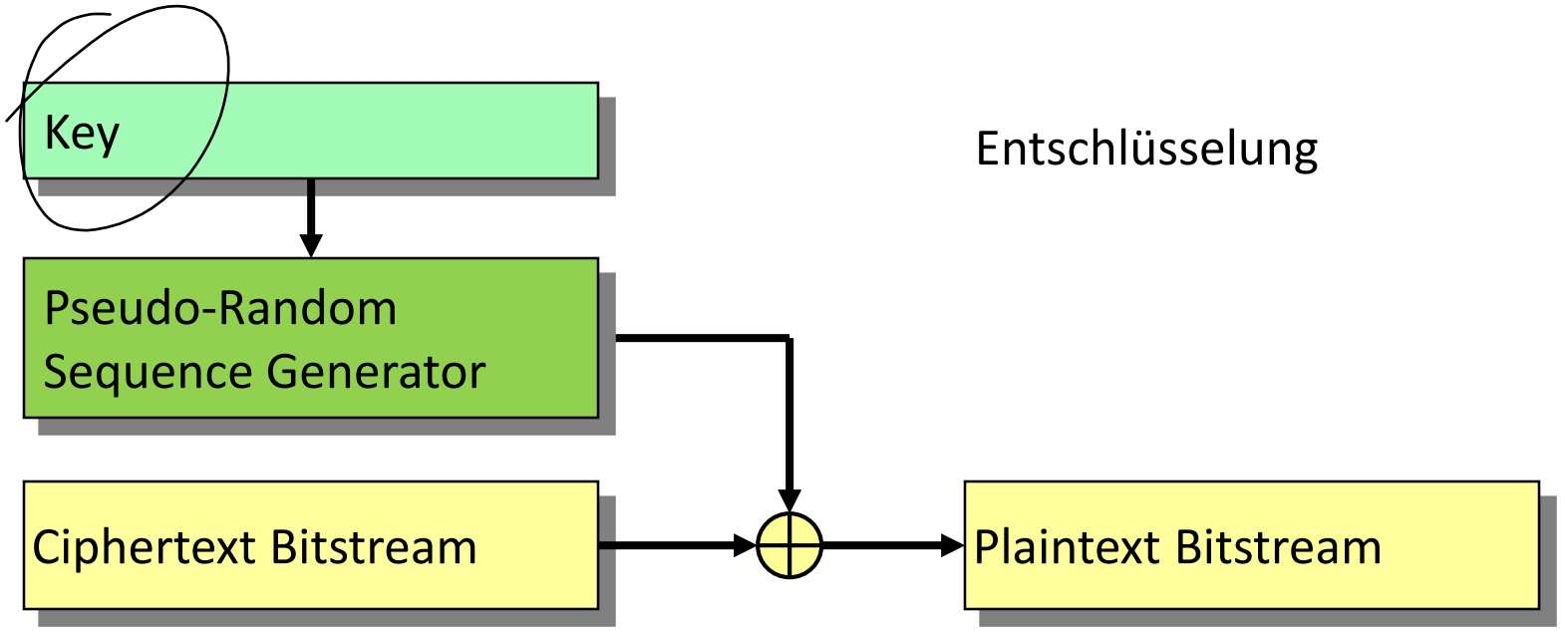
Kap. 7.1
BESCHREIBUNG VON STROMCHIFFREN

Beschreibung von Stromchiffren, encryption



Plaintext Stream	1 1 1 1 1 1 1 1 0 0 0 0 0 0 …
Pseudo-Random Stream	1 0 0 1 1 0 1 0 1 1 0 1 0 0 …
Ciphertext Stream	0 1 1 0 0 1 0 1 1 1 0 1 0 0 …

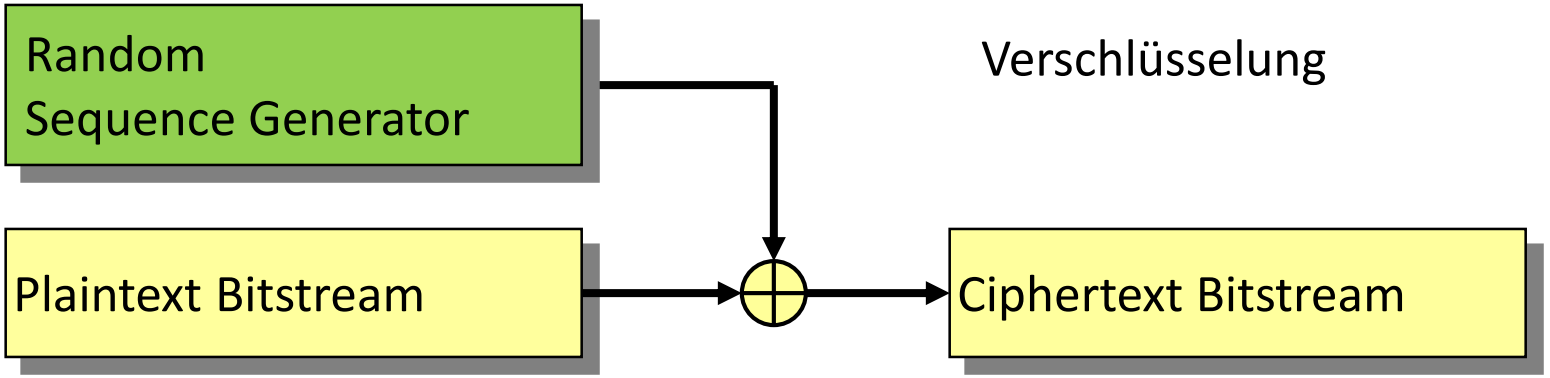
Beschreibung von Stromchiffren, decryption



Ciphertext Stream	0 1 1 0 0 1 0 1 1 1 0 1 0 0 …
Pseudo-Random Stream	1 0 0 1 1 0 1 0 1 1 0 1 0 0 …
Plaintext Stream	1 1 1 1 1 1 1 1 0 0 0 0 0 0 …

Stromchiffren $\overset{?}{=}$ One-Time-Pad (OTP)

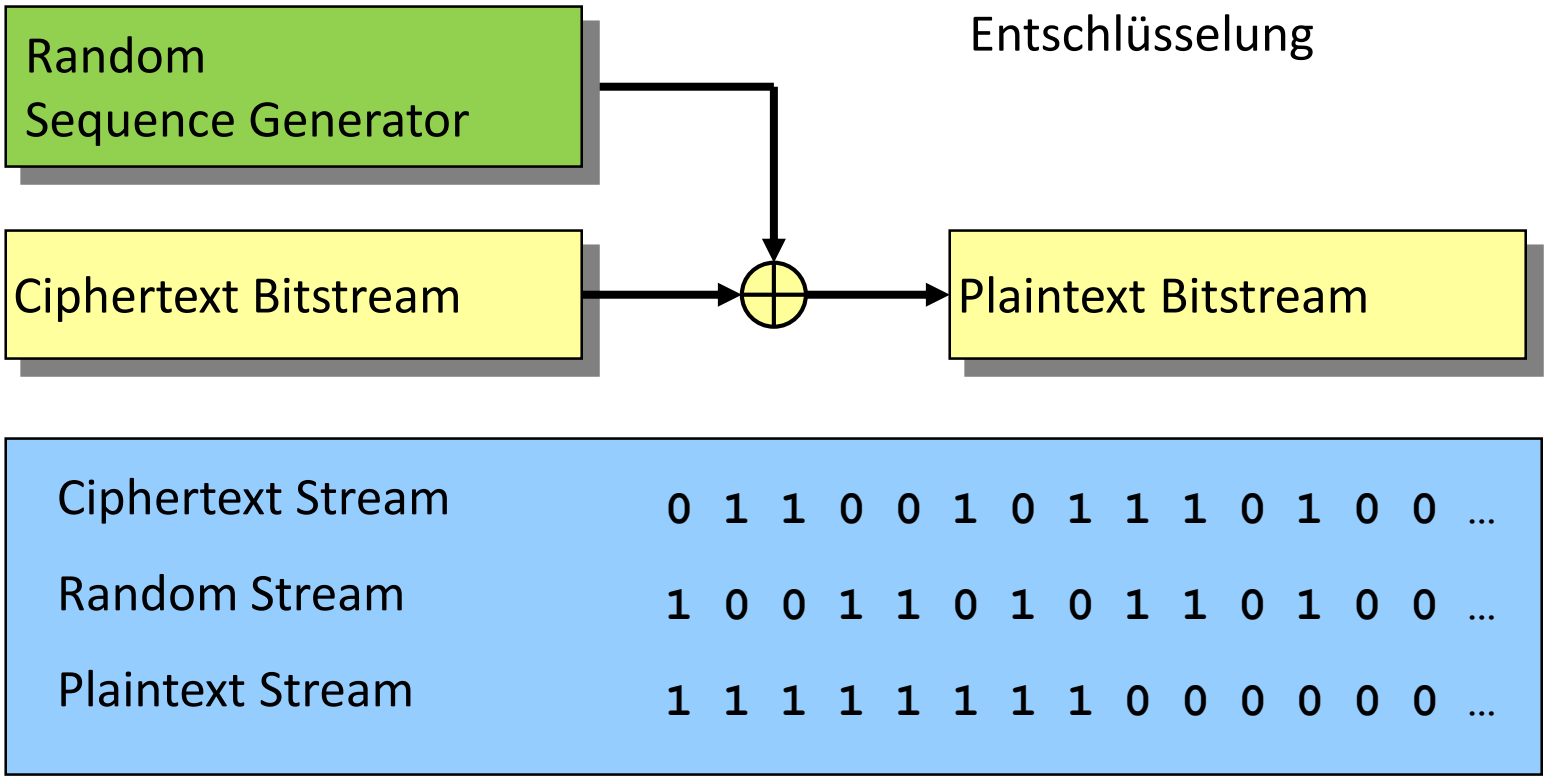
Der One-Time-Pad (OTP) ist eine Stromchiffre mit einem echt zufälligen (und nicht nur pseudo-zufälligen) Schlüsselgenerator.
Das Problem ist nur, wie transportiert man dann diesen echt zufälligen Schlüssel zum Empfänger. Und zudem muss die Schlüssellänge so gross wie der Klartext sein. Dafür ist OTP perfekt sicher (Details, cf. Kap. 28, „Perfekte Sicherheit“ im JS Skript, wobei nur der vorherige Einschub, aber nicht das ganze Kap. 28 prüfungsrelevant sind).



Plaintext Stream	1 1 1 1 1 1 1 1 0 0 0 0 0 0 …
Random Stream	1 0 0 1 1 0 1 0 1 1 0 1 0 0 …
Ciphertext Stream	0 1 1 0 0 1 0 1 1 1 0 1 0 0 …

©Josef Schuler $J\int$ *Skripte*

Stromchiffren $\overset{\Leftarrow}{\nRightarrow}$ *One-Time-Pad (OTP)*



Beschreibung von Stromchiffren, Diverses

Verschlüsselung: $C = M \oplus S$

Entschlüsselung: $C \oplus S = (M \oplus S) \oplus S = M \oplus (S \oplus S) = M \oplus 0 = M$

Charakteristika/Produkte:

Es gibt „nur“ eine Stromchiffre, das XOR; aber die Erzeugung der Pseudo Random Bits, also der Pseudo Random Generator – unten Keygenerator genannt – ist unterschiedlich.

- Der Keygenerator ist …
  - … in gewissen Anwendungen eine Kombination von Schieberegistern → out of scope.
  - … oft (Bankenwelt) ein Blockverschlüssler in einem besonderen Modus (cf. Kap. 8.3.).
  - … oft (Netzwerkwelt) eine spezielle Konstruktion mit Hashfunktionen wie SHA-1.
- Stromchiffren – engl. Stream cipher – sind symmetrische Verfahren
- Vor allem bei Link-Verschlüssler als HW erhältlich.
- Nur als Verschlüssler brauchbar, d.h. ein Stromchiffrierer bietet ***keine Integrität/Authentizität. → siehe auch Beispiel im Skript Kap. 7.1.3!***
- Ein Verändern im Chiffretext wirkt sich nur auf die direkt betroffenen Zeichen aus.
- Wird aber ein Bit aus dem Chiffretext entfernt oder zugefügt, so gerät beim Entschlüsseln – ausser es handelt sich um selbstsynchronisierende Stromchiffrierer – alles ausser „Rand und Band“.

Beschreibung von Stromchiffren, Fort.

Einsatz/Produkte:

1. Link encryption
2. A5 = Verschlüsselung vom Handy zur Funkantenne (A5 ein sich selbstsynchronisierender Stromchiffrierer; Grundlagen von U. Maurer, ETHZ)
3. RC-4 in WLAN Verschlüsselung (WEP), auch die 104 Bit Variante ist schlecht implementiert → besser WPA2 (Einsatz des Blockchiffrier-Algorithmus AES).
4. Neuere Produkte (z.B. Salsa20 oder ChaCha20) kommen in End-to-End Encrypted Message Diensten wie Threema (Salsa20) oder Signal resp. in deren Implementationen wie Wire (ChaCha20) vor.

Und es sei nochmals erwähnt:

Ein Stromchiffre bietet ***keine Integrität/Authentizität → siehe Beispiel im Skript Kap. 7.1.3!***

Aufgabe 7.1

Bearbeiten Sie nun das oben erwähnte Beispiel in Kap. 7.1.3 im Skript „Einführung in die Kryptologie“.

Basis-Test Präsenz 2 – 3 zu Kap. 7.1

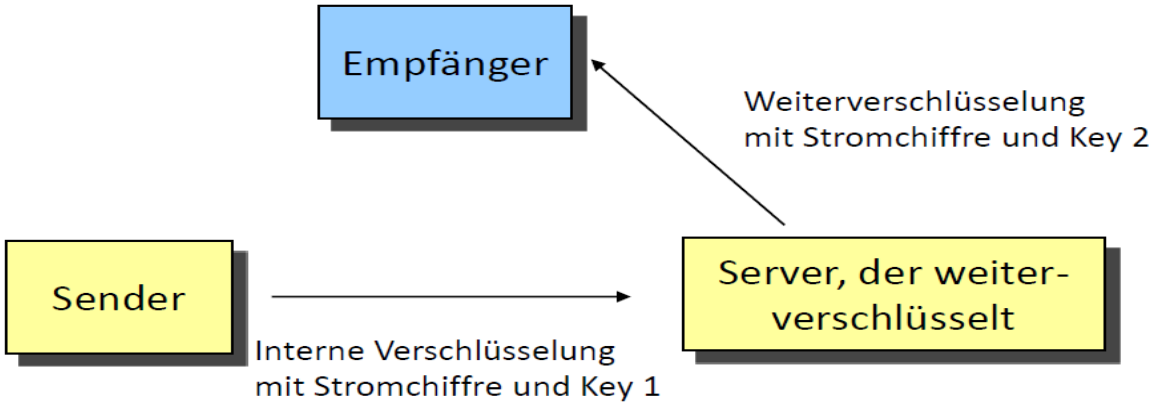
Aussage	Richtig oder falsch?	Begründung
Das wirkliche „Arbeitspferd“ der Kryptologie sind die Stromchiffren.	☐ richtig ☐ falsch	
Bei Stromchiffren ist die Verschlüsselung und die Entschlüsselung exakt die gleiche Operation.	☐ richtig ☐ falsch	
Es gibt mehrere Stromchiffre Verschlüsselungsalgorithmen.	☐ richtig ☐ falsch	
Stromchiffre und One-Time-Pad sind identische Begriffe.	☐ richtig ☐ falsch	
Mit Stromchiffren kann ich neben dem Verschlüsseln auch die Integrität einer Meldung gewähren.	☐ richtig ☐ falsch	
$12 \oplus 36 = 48$	☐ richtig ☐ falsch	

©Josef Schuler $J\int$ *Skripte*

Aufgabe 7.2

xy 17h40

Sie haben nun das folgende Schema einer Weiterverschlüsselung vor sich. Der Sender will dem Empfänger eine verschlüsselte Meldung übermitteln. Zunächst verschlüsselt der Sender die Meldung mit dem Schlüssel Key 1 = 8E. Die Meldung kommt zunächst zu einem Kommunikationsserver, der die erhaltene Meldung mit dem Key 2 = 58 weiterverschlüsselt, ***ohne*** sie vorher zu entschlüsseln. Die Verschlüsselungen sind in beiden Fällen Stromchiffren. Alle Werte sind in HEX. Wenn ihr TR HEX nicht rechnen kann, so benutzen so doch die HEX-XOR Tabelle in Folie 8, resp. im Skript Kap. 7.1.1.



a) Welchen Schlüssel Key 3 muss der Empfänger anwenden, wenn er die Entschlüsselung in einem Schritt durchführen will?

b) Wie lautet die entschlüsselte Meldung, wenn der Empfänger die Meldung = 32 erhalten hat?

Aufgabe 7.3 Beurteilung einer Stromchiffre

a) Welche(s) Schutzziel(e) müssen festgelegt sein, um den Einsatz einer Stromchiffre zu rechtfertigen?
- ☐ Schutz vor unberechtigtem Einfügen einer Meldung.
- ☐ Schutz zum Entdecken, ob die Meldung verändert wurde.
- ☐ Schutz um das Löschen einer Meldung zu entdecken.
- ☐ Schutz, damit sich ein Gegner (Eve) nicht als Absender (Alice) ausgeben kann.
- ☐ Schutz, damit sich ein Gegner (Eve) nicht als Empfänger (Bob) ausgeben kann.
- ☐ Schutz gegen das Abhören der Meldung.
- ☐ Schutz um zu Verhindern, dass der Sender behaupten kann, die Meldung nicht geschickt zu haben.
- ☐ Schutz um zu Verhindern, dass der Empfänger behaupten kann, die Meldung nicht bekommen zu haben.
- ☐ Keine der Angaben ist zutreffend.

b) Mit was für Typen Gegner/Angriffen muss gerechnet werden müssen, um den Einsatz einer Stromchiffre zu rechtfertigen?
- ☐ Es muss hauptsächlich mit Insider Angriffen gerechnet werden.
- ☐ Stromchiffre ist hauptsächlich ein Schutz gegen intelligente Gegner, wie z.B. Hacker, Geheimdienste usw., die einen passiven Angriff durchführen.
- ☐ Stromchiffre ist hauptsächlich ein Schutz gegen intelligente Gegner, wie z.B. Hacker, Geheimdienste usw., die einen aktiven Angriff durchführen.
- ☐ Nicht intelligente Gegner, wie z.B. ein Fehlverhalten (Übertragungsfehler) des Systems oder eines Menschen (Verschreiben).
- ☐ Keine der Angaben ist zutreffend

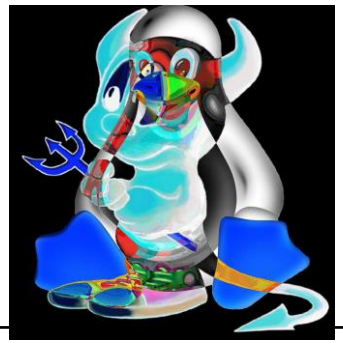
c) Bei der Stromchiffre handelt es sich um was für einen Mechanismus?
- ☐ Asymmetrische Verschlüsselung.
- ☐ Schlüsselloser Kryptographischer Mechanismus
- ☐ MAC-Berechnung
- ☐ Authentizierprotokoll
- ☐ Symmetrische Verschlüsselung
- ☐ Digitale Signatur
- ☐ Diffie-Hellman Schlüsselaustausch
- ☐ Nicht kryptographische Hashfunktion (z.B. eine Prüfziffer)
- ☐ Kryptographische Hashfunktion (z.B. SHA-1)

Aufgabe 7.4 Das Stromchiffre-Puzzle – Sender

Der Sender verschlüsselt zwei Nachrichten M_1 und M_2 mit einer Stromchiffre.

	$i = 1$	$i = 2$
Meldung M_i		
Key K_i		
Chiffrat C_i $C_i = M_i \oplus K_i$		

Aufgabe 7.4 Der Empfänger und das Puzzle

Der Empfänger (das Kryptomodul) erhält zwei verschlüsselte Nachrichten, entschlüsselt sie und zeigt dem Betrachter nachher das folgende Bild:	

a) Was ging hier schief? Beim Sender? Beim Empfänger? Ev. bei beiden etwas? Hinterfragen Sie alle Möglichkeiten, die schief gehen konnten.

b) Betrachten Sie die folgenden Puzzleteile (Sie brauchen nicht alle Teile!) **Tipp:** Schauen Sie an, was der Empfänger (möglicherweise) erhalten hat, und folgern Sie dann, was passiert sein könnte.

• $C_1 = M_1 \oplus K_1$ • $C_2 = M_2 \oplus K_2$ • $C_2 = M_2 \oplus K_1$ • $C_1 = M_1 \oplus K_2$ • $C_1 \oplus C_1 =$ • $C_1 \oplus C_2 =$	• $= (M_1 \oplus M_1) \oplus (M_2 \oplus M_2) =$ • $= (M_1 \oplus K_1) \oplus (M_2 \oplus K_2) =$ • $= (M_1 \oplus K_1) \oplus (M_2 \oplus K_1) =$ • $= (M_1 \oplus M_2) \oplus (M_2 \oplus K_1) =$ • $= (M_1 \oplus K_1) \oplus (M_2 \oplus M_2) =$	• $= M_1$ • $= M_2$ • $= M_1 \oplus M_1$ • $= M_1 \oplus M_2$ • $= M_2 \oplus M_2$

Bemerkung:

Im JS Skript, Kap. 7.1.4 hat es weitere Aufgaben zu den Stromchiffren.

REPETITION AUS DMATH
KAP. 19.5.1 – 19.5.3 DIE FAKULTÄT
= TEIL DER VORBEREITUNG (FLIPPED CLASSROOM)

Definition & Anwendung: Die Fakultät $n!$

$$n! = \begin{cases} 1 \cdot 2 \cdot 3 \cdot \ldots \cdot n = \prod_{i=1}^{n} i \quad \text{für } n > 0, n \in \mathbb{N} \\ 1 \qquad\qquad\qquad\qquad \text{für } n = 0 \end{cases}$$

gesprochen: „n Fakultät“

Wichtig: Man beachte: $0! = 1$

Beispiel 3:

1) $4! \cdot 6! = (4 \cdot 3 \cdot 2 \cdot 1) \cdot (6 \cdot 5 \cdot 4 \cdot 3 \cdot 2 \cdot 1) = 24 \cdot 720 = 17'280$

2) $\frac{7!}{5! \cdot 1!} = \frac{7 \cdot 6 \cdot 5 \cdot 4 \cdot 3 \cdot 2 \cdot 1}{5 \cdot 4 \cdot 3 \cdot 2 \cdot 1 \cdot 1} = \frac{42}{1} = 42$

3) $\frac{7!}{5! \cdot 0!} = \frac{7 \cdot 6 \cdot 5 \cdot 4 \cdot 3 \cdot 2 \cdot 1}{5 \cdot 4 \cdot 3 \cdot 2 \cdot 1 \cdot 1} = \frac{42}{1} = 42$

Anwendung in der Kombinatorik (Anzahl der Permutationen von n Elementen)

Einer der grundlegenden Fragen in der Kombinatorik lautet: „Wie viele Möglichkeiten gibt es n verschiedene Elemente auf n Plätze zu verteilen?“

Antwort: $n!$

Beispiel 4: Auf wie viele Arten können 6 Personen auf 6 unterscheidbaren Stühlen sitzen?

Lösung: $6! = 720$

Weitere Anwendung der Fakultät: Definition des Binomialkoeffizienten $\binom{n}{m} = \frac{n!}{m!(n-m)!}$

Wie schnell wächst $n!$?

Wichtig: $n!$ wächst ziemlich schnell! Und zwar schneller als eine Exponentialfunktion!
Das wird im folg. Video schön gezeigt https://www.youtube.com/watch?v=O55fORXiwxY

Zusammenfassung:

- $n! > a^n$ gilt etwa ab $n > e \cdot a$
- $n! > 3^n$ gilt etwa ab $n > e \cdot 3 \approx 8{,}15$
- $8! = 40'320;\ 3^8 = 6'561;\ 8! > 3^8$
- Je grösser a desto besser $n = e \cdot a$



Beispiel 5:

1) $6! = 720;\ 10! = 3'628'800;\ 13! = 6'227'020'800$ = grösstes $n!$ das ein einfacher TR mit 10-stelliger Anzeige noch genau angeben kann. Nachher muss der TR $n!$ mit der Exponentialdarstellung anzeigen.
2) $14! \approx 8{,}718 \cdot 10^{10}$
3) $60! \approx 8{,}32 \cdot 10^{81}$, das ist ungefähr die geschätzte Anzahl Atome im Weltall.
4) $69! \approx 1{,}71 \cdot 10^{98}$, das ist das grösste $n!$ das ein einfacher TR mit 2-stelliger Exp.darstellung anzeigen kann. $70! \approx 1{,}2 \cdot 10^{100}$ ist eine Zahl mit 101 Dezimalstellen, also $70! > 9{,}9 \ldots \cdot 10^{99}$.
5) $102! \approx 9{,}6 \cdot 10^{161} \approx 10^{162}$, wäre ungefähr die Anzahl der Atome, wenn auf jedem Atom im Weltall ein ganzes Weltall liegen würde.
6) $299! \approx 1{,}02 \cdot 10^{612}$ ist also eine 613-stellige Zahl. Diese Zahl kann der TI-89 (bei anderen grossen TI's wird es ähnlich sein) noch exakt, also alle 613 Zahlen darstellen.
7) $449! \approx 3{,}85 \cdot 10^{997}$ ist nun die grösste $n!$, die der TI-89 (bei anderen grossen TI's wird es ähnlich sein) in der Exponentialdarstellung anzeigen kann.
8) Tools wie Maple könnten aber noch weiterrechnen.

Berechnung von $n!$ bei grossen n

Frage: Wie könnte man $n!$ schneller als mit $1 \cdot 2 \cdot 3 \cdot \cdots \cdot n$ berechnen?

Antwort: Mit der Formel von Stirling.

$$n! \approx \left(\frac{n}{e}\right)^n \sqrt{2 \cdot \pi \cdot n}$$

Beispiel 6:

n	$n!$ direkt berechnet	$n!$ mit Stirling berechnet	Fehler m. Formel v. Stirling
6	$= 720$	$\approx 7{,}1 \cdot 10^2$	$-1{,}4\%$
10	$= 3'628'800$	$\approx 3{,}6 \cdot 10^6$	$-0{,}83\%$
13	$= 6'227'020'800$	$\approx 6{,}187 \cdot 10^9$	$-0{,}64\%$
69	$\approx 1{,}711 \cdot 10^{98}$	$\approx 1{,}709 \cdot 10^{98}$	$-0{,}12\%$
102	$\approx 9{,}61 \cdot 10^{161}$	$\approx 9{,}60 \cdot 10^{161}$	$-0{,}08\%$
299	$\approx 1{,}020 \cdot 10^{612}$	$\approx 1{,}0199 \cdot 10^{612}$	$< -0{,}03\%$
449	$\approx 3{,}8519 \cdot 10^{997}$	$\approx 3{,}8512 \cdot 10^{997}$	$< -0{,}02\%$

Berechnung von $n!$ bei sehr grossen n

Frage: Kann $n!$ auch für sehr grosse n (z.B. $n = 2^{64} \approx 1{,}845 \cdot 10^{19}$ o. $2^{128} \approx 3{,}4 \cdot 10^{38}$) berechnet oder wenigstens die Anzahl der Dezimalstellen angeben werden?

Antwort: Ja, mit dem 10-Logaritmus der Formel von Stirling.

$$\log_{10} n! \approx \log_{10}\left(\left(\frac{n}{e}\right)^n \sqrt{2 \cdot \pi \cdot n}\right) = n \log_{10}\left(\frac{n}{e}\right) + \frac{1}{2}\log_{10}(\pi \cdot n)$$

Beispiel 7:

n	$n!$ direkt berechnet	Anz. Dez.stellen	Anz. Dez.stellen m. log(Stirl.)
6	$= 720 = 7{,}2 \cdot 10^2$	2 + 1 = 3	2 + 1 = 3
10	$= 3'628'800 \approx 3{,}63 \cdot 10^6$	6 + 1 = 7	6 + 1 = 7
13	$= 6'227'020'800 \approx 6{,}23 \cdot 10^9$	9 + 1 = 10	9 + 1 = 10
69	$\approx 1{,}711 \cdot 10^{98}$	98 + 1 = 99	98 + 1 = 99
102	$\approx 9{,}61 \cdot 10^{161}$	161 + 1 = 162	161 + 1 = 162
299	$\approx 1{,}020 \cdot 10^{612}$	612 + 1 = 613	612 + 1 = 613
449	$\approx 3{,}8519 \cdot 10^{997}$	997 + 1 = 998	997 + 1 = 998
$\mathbf{2^{64}}$	--	--	$\approx 3{,}5 \cdot 10^{20}$
$\mathbf{2^{128}}$	--	--	$\approx 1{,}3 \cdot 10^{40}$

$(2^{128})!$ *ist eine Zahl mit* 10^{40} *Stellen*

Frage: Wie lässt sich eine Zahl mit 10^{40} Stellen veranschaulichen?

Antwort: Im Folgenden zwei Versuche.

- Ein 70 kg schwerer Mensch besteht aus ca. $6{,}7 \cdot 10^{27}$ Atomen.
- Alle (zu hoch geschätzten) 10 Milliarden Menschen haben demnach, ca. $6{,}7 \cdot 10^{37}$ Atome.
- Um auf 10^{40} zu kommen, müsste man noch mit dem Faktor 150 multiplizieren.
- Diese Zahl 10^{40} entsprechen der Anzahl der Dezimalstellen von $(2^{128})!$ Also eine 10^{40} stellige Zahl.

Oder:

- Man geht davon aus, dass im Jahr 2025 alle weltweit ausgetauschten Daten die Menge von 175 ZettaByte $= 175 \cdot 10^{21} = 1{,}75 \cdot 10^{23}\ Byte$ erreicht.
- 1 ZettaByte $= 10^{21}$ Byte.
- Wenn wir pro Dezimalzahl 1 Byte brauchen würden, dann könnte man mit ca. 1/200 dieser Jahresdatenmenge zumindest $(2^{64})!$ darstellen.
- Für $(2^{128})!$ bräuchte es aber ca. 10^{19} so viele Daten wie im Jahre 2025. D.h. selbst wenn seit dem Urknall vor 14 Milliarden Jahren $(= 1{,}4 \cdot 10^{10}\ \text{Jahre})$ jedes Jahr so viele Bytes wie im 2025 zur Verfügung stünden, bräuchte es $10^8 = 100\ Mio$ mal so viel wie seit dem Urknall.

REPETITION AUS DMATH
KAP. 19.5.4 – 19.5.7
DER BINOMIALKOEFFIZIENT

= TEIL DER VORBEREITUNG (FLIPPED CLASSROOM)

Definition: Der Binomialkoeffizient $\binom{n}{m}$

$$\binom{n}{m} = \frac{n!}{m! \cdot (n-m)!} \quad f\ddot{u}r\ 0 \le m \le n \quad und \quad n, m \in \mathbb{N}$$

gesprochen: „n tief m“

Beispiel 8:

1) $\binom{11}{4} = \frac{11!}{4! \cdot 7!} = \binom{11}{7} = 330$, weitere Beispiele in JS Skript, Kap. 19.5.4

2) Bei $m = n/2$ ist der Bin.koeff. max., d.h. $\binom{n}{n/2}$ ist max., cf. Pascalsches Dreieck.

Zeile	Pascalsches-Dreieck	Pascalsches-Dreieck mit Binomialkoeffizienten:
0	1	$\binom{0}{0}$
1	1 1	$\binom{1}{0}\binom{1}{1}$
2	1 2 1	$\binom{2}{0}\binom{2}{1}\binom{2}{2}$
3	1 3 3 1	$\binom{3}{0}\binom{3}{1}\binom{3}{2}\binom{3}{3}$
4	1 4 6 4 1	$\binom{4}{0}\binom{4}{1}\binom{4}{2}\binom{4}{3}\binom{4}{4}$
5	1 5 10 10 5 1	$\binom{5}{0}\binom{5}{1}\binom{5}{2}\binom{5}{3}\binom{5}{4}\binom{5}{5}$
6	1 6 15 20 15 6 1	$\binom{6}{0} = 1;\ \binom{6}{1} = 6;\ \binom{6}{2} = 15;\ \binom{6}{3} = 20;\ \binom{6}{4} = 15;\ \binom{6}{5} = 6;\ \binom{6}{6} = 1$

Kombinatorik & der Binomialkoeffizient $\binom{n}{m}$

Allgemeine Merkregel:

Wenn aus n Elementen m Elemente $(n \geq m)$ ausgewählt werden können, und die Reihenfolge keine Rolle spielt, dann gibt es $\binom{n}{m}$ Möglichkeiten.

Beispiel 8, Fortsetzung:

3) $\binom{42}{6}$ beschreibt die Anzahl Möglichkeiten, 6 aus 42 Zahlen auszuwählen. Siehe dazu Lotto 6 aus 42 (Reihenfolge der Ziehung der Zahlen nicht berücksichtigt).
   - 42 Möglichkeiten für 1 Zahl.
   - $42 \cdot 41 = 1722$ für 2 Zahlen mit Berücksichtigung der Reihenfolge, $\frac{42 \cdot 41}{1 \cdot 2} = \binom{42}{2} = 861$ für 2 Zahlen ohne die Reihenfolge zu berücksichtigen.
   - …
   - $42 \cdot 41 \cdot \ldots \cdot 37$ für 6 Zahlen mit Reihenfolge, $\frac{42 \cdot 41 \cdot \ldots \cdot 37}{1 \cdot 2 \cdot \ldots \cdot 6} = \binom{42}{6} = 5'245'786$ für 6 Zahlen ohne Reihenfolge. → Beim CH-Lotto werden die Möglichkeiten wegen der Zusatzzahl noch mit Faktor 6 vergrössert, also $31'474'716$.

4) Analog nun für Ändern der Bits bei einer Blockgrösse von 128 Bit:
   - $\binom{128}{1} = 128$ Möglichkeiten für 1 Bit.
   - $\binom{128}{64} = 2{,}4 \cdot 10^{37}$ Möglichkeiten für die Hälfte, also 64 Bit.

Die Summe aller/einiger Binomialkoeffizienten

Beispiel 8, Fortsetzung:

5) Alle Binomialkoeffizienten zusammengezählt:

$$\sum_{m=0}^{n} \binom{n}{m} = \binom{n}{0} + \binom{n}{1} + \cdots + \binom{n}{n-1} + \binom{n}{n} = 2^n$$

Für n = 6:

$$\sum_{m=0}^{6} \binom{6}{m} = \binom{6}{0} + \binom{6}{1} + \cdots + \binom{6}{5} + \binom{6}{6} = 1 + 6 + 15 + 20 + 15 + 6 + 1 = 64 = 2^6$$

Für n = 128:

$$\sum_{m=0}^{128} \binom{128}{m} = \binom{128}{0} + \binom{128}{1} + \cdots + \binom{128}{128-1} + \binom{128}{128} = 2^{128} \approx 3{,}4 \cdot 10^{38}$$

6) Nur die mittleren 2r Binomialkoeffizienten zusammengezählt, z.B. Anzahl Möglichkeiten $\frac{n}{2} \pm r$ Bit von n Bit zu ändern:

$$\sum_{m=\frac{n}{2}-r}^{\frac{n}{2}+r} \binom{n}{m} = \binom{n}{\frac{n}{2}-r} + \cdots + \binom{n}{n/2} + \cdots + \binom{n}{\frac{n}{2}+r}$$

Die Summe aller/einiger Binomialkoeffizienten

Beispiel 8, Fortsetzung:

6) Fort.: Für n = 128 und r = 15, also die mittleren 30 Binomialkoeffizienten zusammengezählt, z.B. Anzahl Möglichkeiten zwischen 49 – 79 Bit von 128 Bit zu ändern:

$$\sum_{m=\frac{n}{2}-r}^{\frac{n}{2}+r}\binom{n}{m} = \sum_{m=\frac{128}{2}-15}^{\frac{128}{2}+15}\binom{128}{m} = \sum_{m=49}^{79}\binom{128}{m} = \binom{128}{49}+\cdots+\binom{128}{64}+\cdots+\binom{128}{79} \approx 3{,}38\cdot 10^{38}$$

Damit kann der Anteil dieser mittleren 30 Binomialkoeffizienten berechnet werden = Anteil aller Möglichkeiten zw. 49 – 79 Bit von 128 Bit zu ändern:

$$\frac{\sum_{m=49}^{79}\binom{128}{m}}{2^{128}}\cdot 100 \approx \frac{3{,}38\cdot 10^{38}}{3{,}4\cdot 10^{38}} \approx 99{,}4\%$$

Resp. In nur 0,6% aller Fälle werden weniger als 49 oder mehr 79 Bits geändert.

7) Für n = 128 & r = 10. Ca. 94% der Werte liegen im Bereich +/- 10 um die Mitte, d.h. $54 \le m \le 74$

$$\sum_{m=54}^{74}\binom{128}{m} = \binom{128}{54}+\cdots+\binom{128}{64}+\cdots+\binom{128}{74} \approx 3{,}2\cdot 10^{38} \approx 0{,}94\cdot 2^{128}$$

Resp. In nur 6% aller Fälle werden weniger als 54 oder mehr 74 Bits geändert.

Bemerkung:

Solche Berechnungen werden bei den Blockchiffren und Hashfunktionen konkret angewandt.

Binomialkoeffizienten Resumée

Für n > 100 kann grob Folgendes zusammengefasst ausgesagt werden:

- Die mittleren 20% der Binomialkoeffizienten zusammengezählt ergeben > 99% der Gesamtsumme von 2^n.
- Die äusseren je 25% der Binomialkoeffizienten zusammengezählt ergeben nur einen Bruchteil von einem Promille der Gesamtsumme von 2^n.

$$\sum_{m=0}^{n} \binom{n}{m} = \underbrace{2^n}_{=100\%}$$

$$= \underbrace{\binom{n}{0} + \binom{n}{1} + \cdots +}_{<0{,}01\%} \ldots + \underbrace{\binom{n}{\frac{n}{2}-r} + \cdots + \binom{n}{\frac{n}{2}} + \cdots + \binom{n}{\frac{n}{2}+r}}_{>99\%} + \cdots \underbrace{+ \cdots + \binom{n}{n-1} + \binom{n}{n}}_{<0{,}01\%}$$

Bemerkungen:

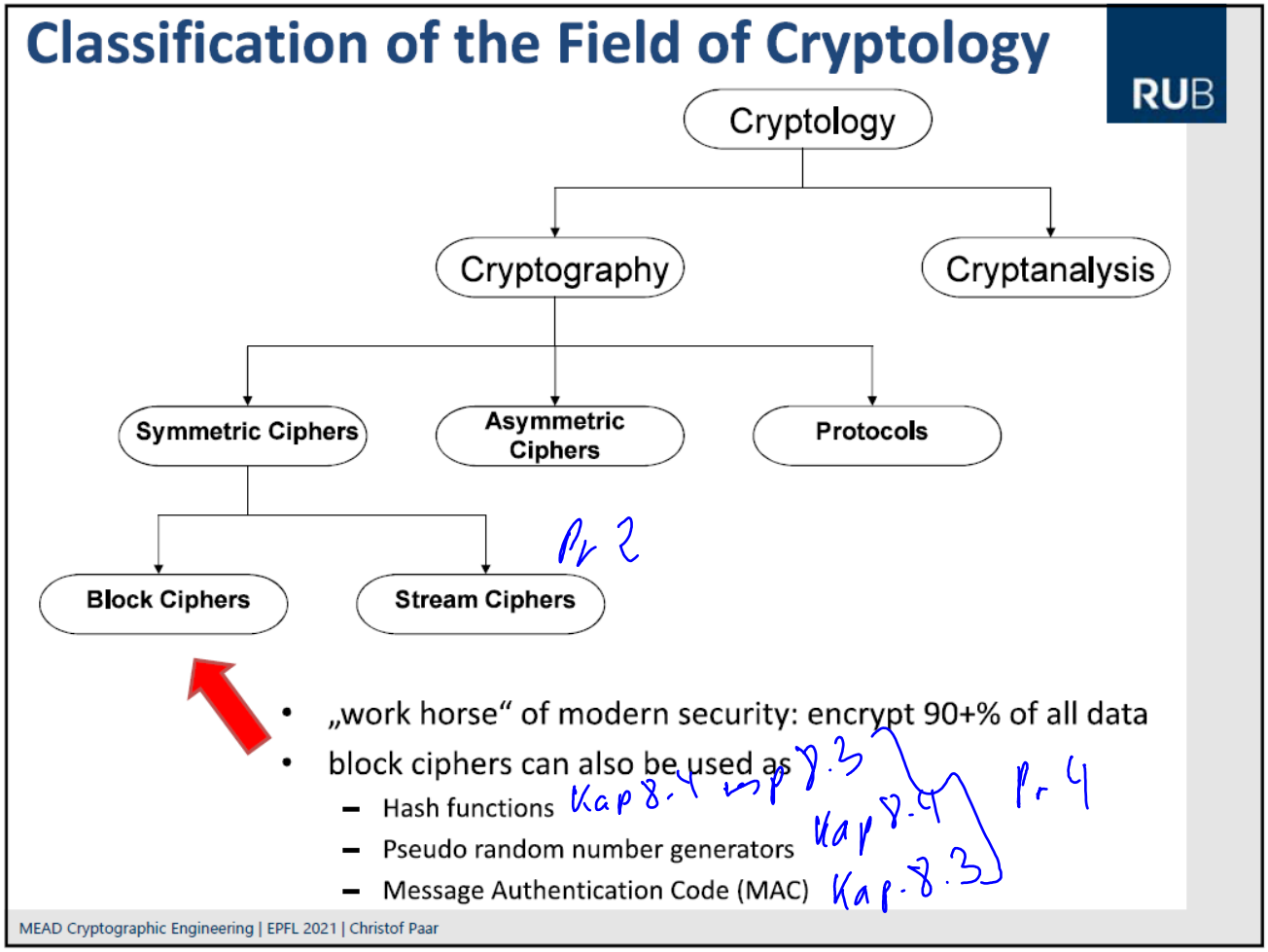
Je grösser n (z.B. bei den Hashfunktionen mit n = 384 oder n = 512) akzentuiert sich dass wie folgt:

- Die mittleren 15% der Binomialkoeffizienten zusammengezählt ergeben > 99% der Gesamtsumme von 2^n.
- Die äusseren je 30% der Binomialkoeffizienten zusammengezählt ergeben nur einen winzigen Bruchteil von einem Promille der Gesamtsumme von 2^n.
- In der Präsenz 4 werden diese Effekte noch detaillierter ausgeführt.

Kap. 7.2
BESCHREIBUNG VON BLOCKCHIFFREN

Blockchiffre das „work horse“ der Sicherheit

C. Paar zeigt im folgenden Bild sehr schön die Bedeutung der Blockchiffren.



<u>Wichtige Ergänzung:</u>

Blockchiffren sind deutlich resistenter gegenüber Quantencomputer als asym. Verfahren.
Man entwickelt z.Z. „nur“ Post Quantum Algorithmen für asymmetrische Verfahren.

Allgemeine Beschreibung von Blockchiffren

Übliche Blockgrössen:

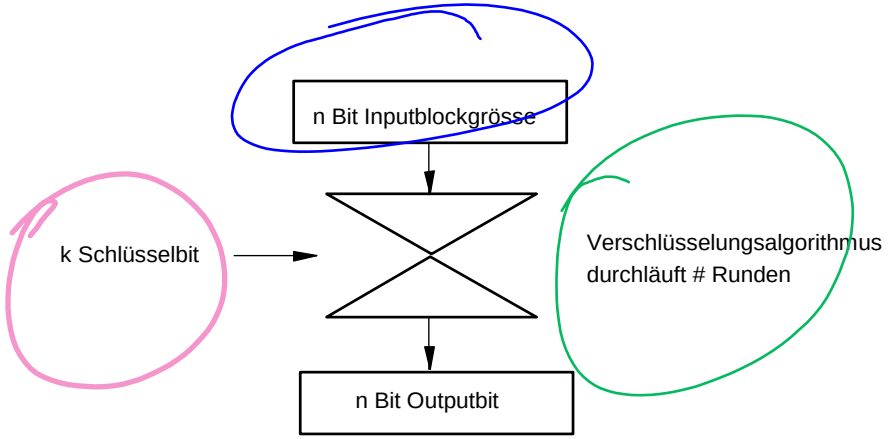
- n = 64, 128 Bit

Anzahl Runden:

- # = 10, 12, 14, 16 u.a.

Übliche Schlüssellängen:

- k = 56 (DES), 112 (3DES_2key)
- k = 128 (AES), 168 (3DES_3key)
- k = 192, 256 (AES) Bit



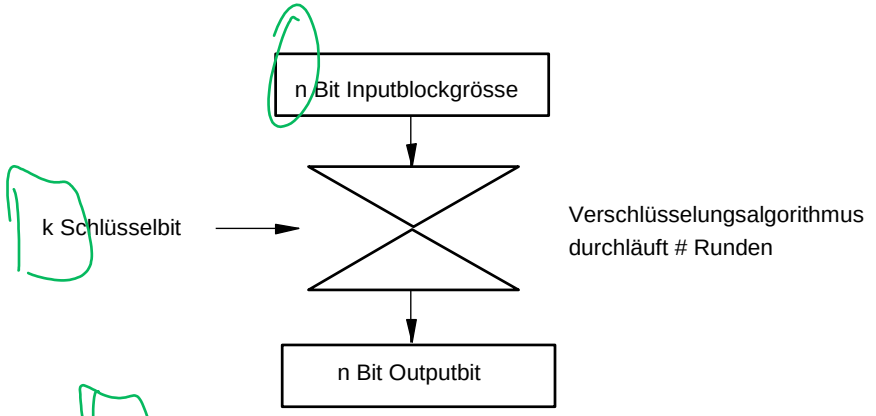
Charakteristika:

- Symmetrische Verfahren
- Blockgrösse, Schlüssellängen und Rundenanzahl (= Teilschritte) sind die Kerngrössen
- Meistens in HW und in SW erhältlich
- Als Verschlüssler, Schlüsselerzeuger oder als Authentizierer verwendbar
- Verschiedene Betriebsarten (siehe Kap. 8)

- **Einsatz:**
  - Bankenwelt
  - Internet (SSL, TLS, IPSec, PGP)
  - ePassport
  - Messager Dienste (z.B. WhatsApp oder Telegramm) → ***Achtung:*** in anderen Diensten werden Stromchiffrierer verwendet, cf. Kap. 7.1, resp. Folie 17.

Mathematische Beschreibung von Blockchiffren

Die Verschlüsselung mit einer Blockchiffre mit Input M der Länge n Bit, Schlüssel K der Länge k Bit und Output C der Länge n Bit kann wie folgt mathematisch beschrieben werden:



$$E: \{0,1\}^n \times \{0,1\}^k \to \{0,1\}^n$$
$$M \times K \to C = E(M,K)$$

Und für die Entschlüsselung gilt:

$$D: \{0,1\}^n \times \{0,1\}^k \to \{0,1\}^n$$
$$C \times K \to M = D(C,K) = D(E(M,K),K)$$

Oder anders formuliert:

- Jedes der 2^n Elemente des Inputs wird in genau eines der 2^n Elemente des Outputs abgebildet. Und umgekehrt, also jedes der 2^n Elemente des Outputs haben genau eines der 2^n Elemente des Inputs als Urbild.
- Mathematisch heisst die Funktionen E und D sind bijektiv.
- Oder anders formuliert heisst das, dass E und D nichts anderes als Permutationen der 2^n Elemente sind.
- Diese Permutationen werden jeweils durch den Schlüssel K parameteriesiert.

Blockchiffren als Menge von Permutationen

Beispiel 9:

Nehmen wir an, wir hätten eine Blockchiffre konstruiert mit der Blockgrösse 4 Bit. D.h. der Input und der Output sind je ein HEX-Zeichen, d.h. $M = C = \{0, 1, \dots, F\}$. Die Schlüsselgrösse dieser Blockchiffre sei k = 8 Bit, also 2 HEX-Zeichen, d.h. $K = \{00, 01, \dots, 0F, 10,11,12, \dots, 1F, \dots, FF\}$.

Somit

$$E: \{0,1\}^4 \times \{0,1\}^8 \to \{0,1\}^4$$
$$M \times K \quad \to C = E(M, K)$$

Es gibt also:

- $2^4 = 16$ mögliche Inputs und Outputs, d.h. $M = C = \{0, 1, \dots, F\}$.
- $2^8 = 256$ mögliche Schlüssel, d.h. $K = \{00, 01, \dots, 0F, 10,11,12, \dots, 1F, \dots, FF\}$.
- Für jeden der $2^8 = 256$ Schlüssel werden also die 16 Elemente des Inputs auf die 16 Elemente des Outputs abgebildet. Also für jeden der 256 Schlüssel gibt es eine Permutation von 16 Elementen, also 256 Permutationen.
- z.B. siehe nächste Folie.

Blockchiffren als Menge von Permutationen

Beispiel 9, Fortsetzung:

Enc		0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
00		1	0	3	2	5	4	7	6	9	8	B	A	D	C	F	E
01		2	3	0	1	6	7	4	5	A	B	8	9	E	F	C	D
02		9	8	B	A	D	C	F	E	1	0	3	2	5	4	7	6
03		4	5	6	7	0	1	2	3	C	D	E	F	8	9	A	B
…																	
FE		A	B	8	9	E	F	C	D	2	3	0	1	6	7	4	5
FF		D	C	F	E	9	8	B	A	5	4	7	6	1	0	3	2

- Es gibt $16! = 20'922'789'888'000 \approx 2{,}09 \cdot 10^{13}$ Möglichkeiten (= Anzahl möglicher Permutationen) 16 Elemente auf 16 Plätze zu verteilen.
- $\frac{2^8}{(2^4)!} = \frac{256}{16!} \approx \frac{256}{2{,}09 \cdot 10^{13}} \approx \frac{1}{8{,}1 \cdot 10^{10}} \approx 10^{-11}$, der Bruchteil ist also ca. 1 : 100 Milliarden.
- Es kommt natürlich auf das Design der Blockchiffre an, welche 256 Permutationen von allen möglichen $2{,}09 \cdot 10^{13}$ ausgewählt werden.

Aufgabe 7.5

Führen Sie nun die gleichen Überlegungen und Berechnungen durch für eine Blockchiffre mit Input-/Outputblockgrösse von n = 8 Bit und der Schlüsselgrösse von k = 32 Bit.

Wie sähe es aus, wenn die Schlüsselgrösse k = 256 Bit wäre?

Blockchiffren als Menge von Permutationen

Die Blockchiffre ist somit also nichts anderes als eine Permutation von 2^n Elemente, also eine Anordnung von 2^n Elemente auf 2^n Plätze.
Jeder der 2^k Schlüssel parametrisiert genau eine dieser Permutationen.

Es gibt nun:

- 2^k mögliche Schlüssel.
  - Für k = 256 Bit folgt $2^k = 2^{256} \approx 10^{77}$, d.h. ca. die Anzahl der Atome im Weltall.
- $(2^n)!$ mögliche Permutationen.
  - Für n = 128 Bit folgt $(2^n)! = (2^{128})! \approx 10^{(10^{40})}$, d.h. eine 10^{40} stellige Zahl.

Somit wird nur ein ***ganz*** kleiner Bruchteil der möglichen Permutationen verwendet:

- Nämlich nur $\frac{2^k}{(2^n)!}$
- Bei AES mit n = 128 Bit Input-/Outputgrösse und k = 256 Bit Schlüsselgrösse wäre diese $\frac{2^{256}}{(2^{128})!} \approx \frac{10^{77}}{10^{(10^{40})}}$, also wirklich ein winziger Bruchteil
- Jede Blockchiffre verwendet andere Permutationen.
- Übergeordnete Eigenschaften & Designkriterien werden weiter hinten besprochen.

Bemerkung:

Auf die Beschreibung der Umsetzung bei bestimmten Blockchiffren (AES, DES usw.) wird in diesem Modul verzichtet. Details könnten in [CP-2] in den Kap. 3 & 4 studiert werden.

Notwendige Eigenschaften von Blockchiffren

- ***Grosser Schlüsselraum*** ist eine notwendige (aber nicht hinreichende Bedingung), dass eine (Block-)Chiffre sicher ist.

Aufgabe 7.6

Setzen Sie ***alle*** möglichen Zeichen aus der Menge $\{\Rightarrow, \Leftrightarrow, \Leftarrow, \nRightarrow, \nLeftrightarrow, \nLeftarrow\}$ zwischen den nachfolgenden zwei Aussagen ein

$$\textit{Grosser Schlüsselraum} \quad \left\{ \qquad \right\} \quad \textit{Die Blockchiffre ist sicher}$$

Claude Shannon (cf. JS Skript Kap. 2.2.4), der Begründer der modernen Informationstheorie (Entropie, cf. JS Skript & Präsenz 1, Kap. 5.1), hat auch die weiteren grundlegenden Eigenschaften für starke Chiffren formuliert:

- ***Konfusion*** besagt, dass der Zusammenhang zwischen Klartext, Chiffretext und Schlüssel möglichst komplex gestaltet werden muss. Sowohl bei DES wie AES wird das mit sog. Substitutionstabellen realisiert.
- ***Diffusion*** besagt, dass jedes Chiffretextzeichen von möglichst vielen Klartextzeichen und dem gesamten Schlüssel abhängen. Bei DES und AES wird das mit unterschiedlichen Operationen realisiert.

Grund: Die sog. Differential Cryptoanalysis, sie wird in diesem Modul ***nicht*** betrachtet!

Folgerung aus den notwendigen Eigenschaften

Anbei aus [CP-D], Kap. 3.1.1, S. 65 unten zitiert:
Moderne Blockchiffren besitzen hervorragende Diffusionseigenschaften. Auf Ebene der Chiffre bedeutet dies, dass die Änderung eines Bits im Klartext die Änderung von ***durchschnittlich*** der Hälfte aller Ausgabebits zur Folge hat.

Horst Feistel (cf. JS Skript Kap. 7.2.9), der Begründer der Feistel-Chiffre, präzisiert diese Diffusionseigenschaft sinngemäss wie folgt:

- ***Der Avalanche-Effekt = Lawineneffekt*** besagt, dass…
  - … die Änderung eines Bits im Klartext ***bei gleichem Schlüssel*** die Änderung von ***durchschnittlich*** der Hälfte aller Ausgabebits zur Folge hat.
  - … resp. die Änderung eines Bits im Schlüssel ***bei gleichem Klartext*** die Änderung von ***durchschnittlich*** der Hälfte aller Ausgabebits zur Folge.

Horst Feistel formulierte ebenso das schärfere Kriterium.

- ***Das strikte Avalanche-Kriterium*** verschärft die obige Aussage „die Änderung von ***durchschnittlich*** der Hälfte aller Ausgabebits“, wie folgt: „jedes Ausgabebit wird mit Wahrscheinlichkeit ½ geändert“.

Bemerkungen:

1) Diese Krit. werden u.a. mit einer genügend grossen Zahl von Verschlüsselungsrunden erreicht.
2) Beim Design von Blockchiffre sind noch viele weitere Eigenschaften zu berücksichtigen. An dieser Stelle werden sie nicht weiter erörtert und sind auch in den allermeisten Lehrbüchern nicht zu finden. In [FR] werden sie in Kap. 4.1 auf 30 Seiten vertieft. [JS1] enthält eine Zus.fung.

Folgerung aus den notwendigen Eigenschaften

Aufgabe 7.7

Setzen Sie ***alle*** möglichen Zeichen aus der Menge $\{\Rightarrow, \Leftrightarrow, \Leftarrow, \nRightarrow, \nLeftarrow, \nLeftrightarrow\}$ zwischen den nachfolgenden zwei Aussagen ein

$$\textit{Striktes Avalanche-Kriterium} \quad \left\{ \qquad \right\} \quad \textit{Avalanche-Effekt}$$

Beispiel 10:

Nehmen wir eine Blockchiffre mit Blockgrösse 128 Bit. Beim Ändern eines Bits im Klartext (bei gleichem Schlüssel) resp. Ändern eines Schlüsselbits (bei gleichem Klartext) würden immer die ersten 15 Bits des Outputs nicht geändert, dafür aber alle 15 letzten Bits. Die „mittleren" 98 Bits würden je mit Wahrscheinlichkeit ½ geändert. Dann würde im stat. Mittel die Hälfte der Outputbits geändert (Avalanche-Effekt). Das strikte Avalanche-Kriterium wäre aber nicht erfüllt, da nicht jedes Outputbit mit Wsk. ½ geändert hat.

Bemerkung: Oft wird – auch in diesem Modul – nur mit dem Avalanche-Effekt argumentiert.

Frage:

Haben wir Implikationen der Form …

$$\textit{Jedes Bit ändert mit Wsk. ½} \left\{ \begin{matrix} \Rightarrow \\ \nLeftrightarrow \end{matrix} \right\} \textit{Im stat. Mittel ändert die Hälfte der Bits}$$

… schon einmal betrachtet?

16h05

Antwort:

Ja, in der Präsenz 1, cf. Kap. 5.4 „Einführung in die Hashfunktionen". Dort wurde mit den Eigenschaften A) und B) die analoge Betrachtung durchgeführt.

Mythos „komplizierter = sicherer“

Man könnte ja meinen, je komplizierter eine Blockchiffre aufgebaut ist, desto sicherer ist sie. Das stimmt leider nicht. Dazu ein paar Fakts aus der Historie:

- Seit Beginn Kritikpunkt am DES: die Schlüsselgrösse mit 56 Bit ist zu klein.
- Ebenso wurde festgestellt (und auch kritisiert), dass gewisse Elemente (z.B. die Substitutionsboxen oder S-Boxen) im DES nicht ideal sind. Man hätte idealere Boxen nehmen können.
- So publizierte die österreichische Sicherheitsexpertin Ingrid Schaumüller-Bichl 1981 den GDES (Generalized DES), dieser verwendete idealere S-Boxen und war generell viel komplizierter aufgebaut.
- Ca. 1990 publizierten Eli Biham und Adi Shamir die „Differential Cryptoanalysis“ und kamen u.a. zu folgenden Erkenntnissen:
  - Die „Differential Cryptoanalysis“ wäre bei einem DES mit mehr als 56 Bit schneller als Brute Force → im Nachhinein haben die Entwickler zugegeben, dass sie die „Differential Cryptoanalysis“ schon gekannt und die meisten Resultate auch erhalten haben. Das hat die Schlüssellänge (eben nur 56 Bit) und Wahl der S-Boxen stark beeinflusst.
  - Würde man andere – als eben die nicht idealen S-Boxen – nehmen, dann würde der DES viel schneller geknackt.
  - Insbesondere der – kompliziertere – GDES konnte mit der „Differential Cryptoanalysis“ um Grössenordnungen schneller geknackt werden.

Berechnungen mit Binomialkoeffizienten

Im Zusammenhang der Eigenschaften resp. Design von Blockchiffren kann nun z.B. gefragt werden, wie gross ist die Wahrscheinlichkeit, dass zwischen r und s Bits geändert werden.

Diese Fragestellungen und die entsprechenden Berechnungen wurden in den mathematischen Repetitionen durchgeführt.

Bemerkung:

In Ilias ist ein Maple-Programm hochgeladen, um eigene Werte zu berechnen.

Mit dem Demotool CRYPTICS: https://cryptics.dschenker.ch wird versucht diese Effekte zu veranschaulichen.

Achtung:

- Im Tool CRYPTICS wird mit den Avalanche Kriterium und nicht mit dem strikten Avalanche Kriterium argumentiert. Das würde in einer Nachfolge-version geändert.
- Im folgenden Beispiel sind pro Fall (= Zeile) zwei Beispiele aufgeführt.

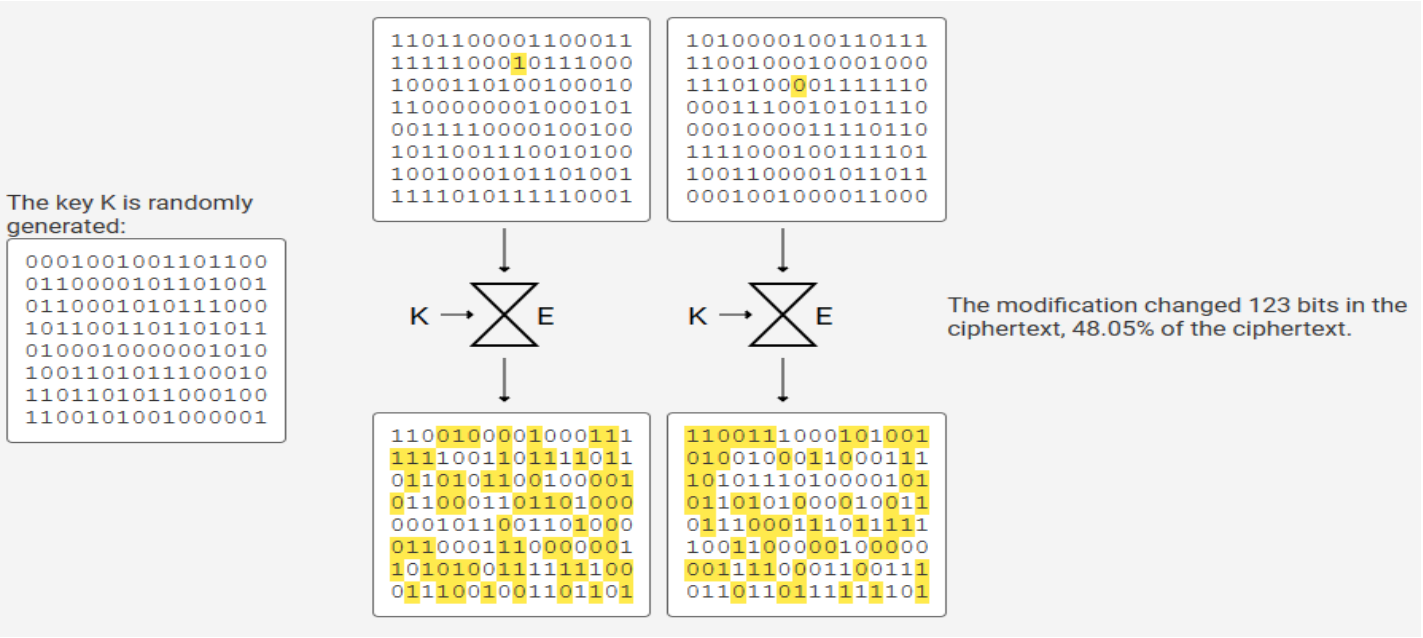
Demo mit CRYPTICS https://cryptics.dschenker.ch

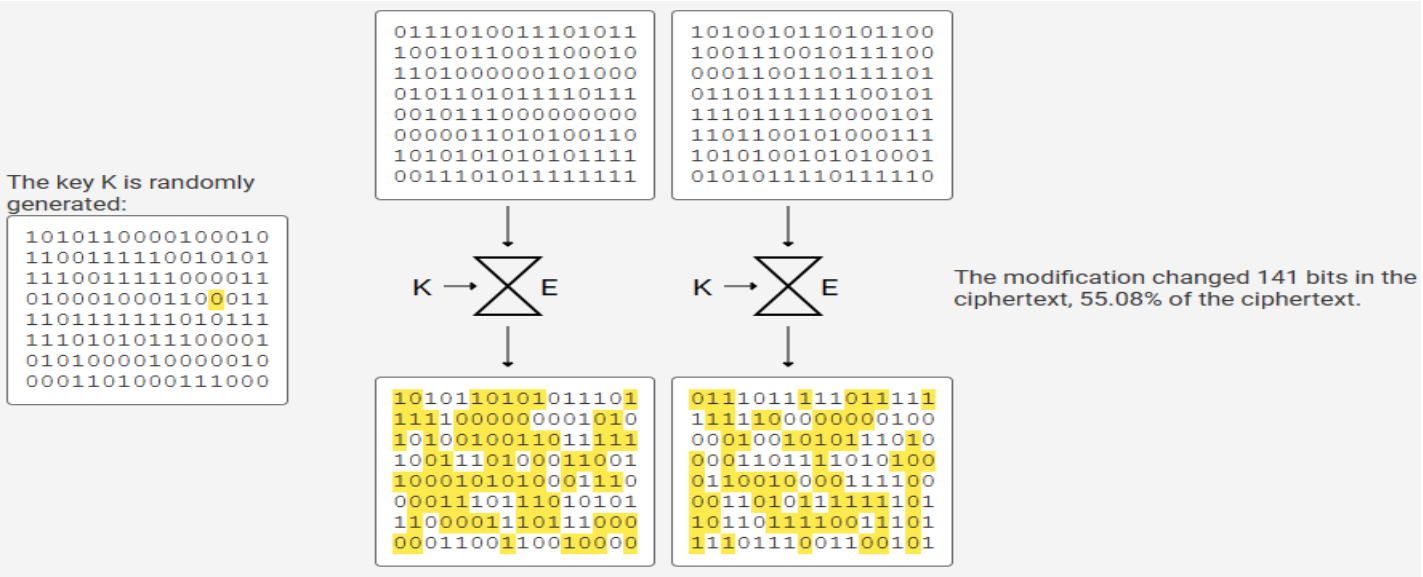
Beim Ändern eines Inputbits, wird bei gleichem Schlüssel – im statistischen Mittel die Hälfte der Outputbits geändert.

Hat noch ein paar Kinderkrank-heiten!!!

Z.B. reagiert z.T. nicht bei Anklicken eines Bits!

Beim Ändern eines Schlüsselbits, wird bei gleichem Input – wiederum im statistischen Mittel – die Hälfte der Outputbits geändert.





Produkte von Blockchiffren

Produkt	Blockgrösse	Schlüssellänge	# Runden	Bemerkungen
DES	64	56	16	• Darf nur noch als Triple DES verwendet werden.
3DES mit 2 Schlüsseln	64	112	16	• Ist – gemäss NIST – seit 2009 nicht mehr erlaubt. • Sicherheit ist deutlich kleiner als 112 Bit (Chosen Plaintext Attacke), nämlich nur ca. 57 – 60 Bit. • Also nicht mehr verwenden.
3DES mit 3 Schlüsseln (#)	64	168	16	• Die effektive Schlüsselstärke ist – bez. einer Meet in the Middle Attacke – aber ***nur*** 112 Bit. • Ist – gemäss NIST – bis 2030 erlaubt, da bis dann 112 Bit genügen sollten.
IDEA	64	128	8	• Blockverschlüssler in PGP. • Schnell • Hat sich wegen den Lizenzgebühren beim kommerziellen Einsatz nicht durchgesetzt.
SAFER-SK128	64	128	6 – 16	• In HW und SW schnell • Gratis • Designed für 8- und 16-Bit Arithmetik (für Chipkarten)
RC-5 (*)	64	Bis 2048 128 (Default)	12	• War Vorbild zum AES.
AES(****)	128 (**)	128, 192 oder 256	10, 12, 14 (***)	• Nachfolger von DES • Standardisiert
PRESENT	64	80/ 128	31	• Als Lightweigth Blockcipher standardisiert. • Wird in div. Kleingeräten verwendet.
Andere (##)				• Nur Standards benutzen.

©Josef Schuler $J\int Skripte$

Produkte von Blockchiffren, Bemerkungen

(#) Der 3DES wird immer noch häufig eingesetzt (z.B. standardmässig bei ePassport). Der Grund liegt darin, dass in einer Chipkarte mit DES-Chip der DES i.d.R. 2 mal schneller ist als ein AES.

(*) Im Prinzip ist RC-5 voll parametrisierbar in (Blocklänge/Schlüssellänge/#Runden)

(**) 128 Bit Blockgrösse ist im Standard so festgelegt. Der Algorithmus (Rijndael = Abkürzung der Namen der zwei Kryptologen Vincent Rijmne und Joan Daemen) liesse aber jedes Vielfache von 32 zu.

(***) Abhängig von der Schlüsselgrösse.

(****) Es ist zwischen dem Standard AES u. dem Algorithmus Rijndael zu unterscheiden.

(##) Viele weitere Algorithmen, siehe z.B. bei der Aufzählung der AES Finalisten. Es gibt aber noch unzählige weitere.

Schlussbemerkung = sinngemässe Wiedergabe aus [RO1]:

Es gibt (in den meisten Fällen) keinen Grund etwas anderes als den AES zu nehmen, so lange dieser als sicher gilt. Z.Z. gibt es keine Hinweise, dass AES in nächster Zeit unsicher würde. AES dominiert das Feld der E2EE (End-to-End Encrypted Message Dienste wie WhatsApp u.v.m.).
Das deckt sich auch mit der [BSI_TR02] Empfehlung.
Eine Ausnahme bilden Kleingeräte, für die PRESENT als Lightweight Blockcipher standardisiert ist.

ISO-18033 – Encryption Algorithms – Block Ciphers

This part of ISO/IEC 18033 specifies block ciphers. A block cipher maps blocks of *n* bits to blocks of *n* bits, under the control of a key of *k* bits. A total of seven different block ciphers are defined. They are categorized in Table 1.

Table 1 — Block ciphers specified

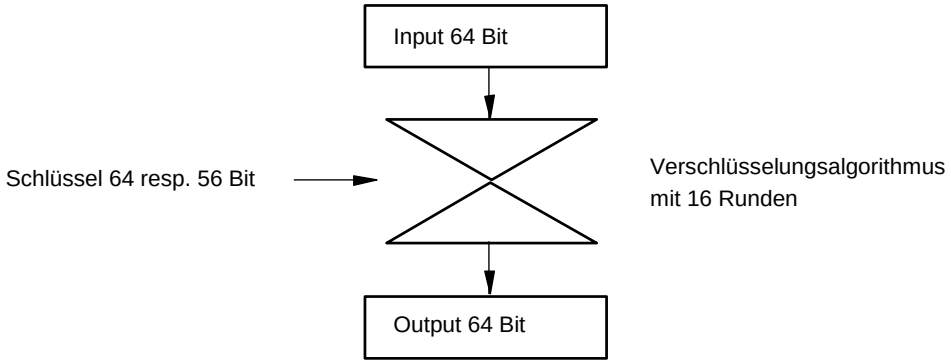
Block length	Algorithm name	Key length
64 bits	TDEA (4.2)	128 or 192 bits (@)
	MISTY1 (4.3)	128 bits
	CAST-128 (4.4)	
	HIGHT (4.5)	
128 bits	AES (5.2) (#)	128, 192 or 256 bits
	Camellia (5.3)	
	SEED (5.4)	128 bits

(@)Beim TDEA (Tripple-DES mit 2 resp. 3 Schlüsseln) ist die wirkliche Schlüsselgrösse natürlich nur 112, resp. 168 Bit. Die Angabe im Standard ist zumindest verwirrend, um nicht einfach „falsch“ schreiben zu müssen. Der DES hat nur eine effektive Schlüssellänge von 56 Bit, obwohl 64 Bit eingegeben werden. Jedes 8-te Bit ist ein Parity-Bit → cf. nachfolgende Folien.

Bemerkung: Der DES (Data Encryption Standard) wird auch mit DEA (Data Encryption Algorithm) abgekürzt, entsprechend 3DES oder 3-DES mit TDEA.

(#) Sowohl das BSI in [BSI_TR02] wie auch Rolf Oppliger in [RO1] empfehlen den AES zu verwenden.

Beispiel 11: DES (Data Encryption Standard)

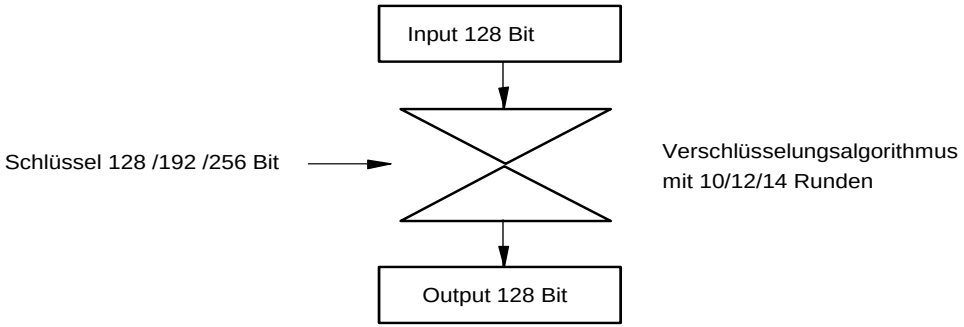


Es wird ein 64 Bit Schlüssel eingegeben, aber jedes 8-te Bit ist ein Parity-Bit, das je nach Algorithmus geprüft oder ignoriert wird. Wenn das Parity-Bit nicht geprüft wird, kann man sagen, dass es zu jedem Schlüssel weitere 255 „identische“ Schlüssel gibt. Alle 256 Schlüssel unterscheiden sich nur in den Parity-Bits. Somit beträgt die effektive Schlüsselgrösse nur 56 Bit.

Fakts zum DES (Data Encryption Standard), tw. aus [CP21] zitiert:

- Einer der am meisten verwendeten Blockchiffren in den 1980-er bis 2000-er Jahre.
- DES ist eine sogenannte Feistelchiffre (der Begriff wird in diesem Modul nicht weiter erörtert, cf. Kap. 7.2.9 in JS Skript „Einf. in die Krypt.“, ***nicht*** Prüfungsstoff). ***Vorteil:*** „Encrypt“ & „Decrypt“ sind bis auf die Reihenfolge der Verwendung der Schlüsselbits gleich.
- 3DES ist gemäss NIST noch bis 2030 erlaubt und wird für epassport und banking verwendet.
- 1998 ist das US Patent abgelaufen.
- DES beeinflusste viele weitere Blockchiffrealgorithmen.
- Der Single-DES gilt als geknackt (2008: innerhalb von 6 Tagen mit 10‘000 $ Kosten)
- DES ist insbesondere wegen (unnötigen) Permutationen in HW viel schneller als in SW. Z.B.:
  - In (Spezial-)HW > 10 Gbit/sec.
  - In (hoch optimierter) SW ca. 400 Mbit/sec. Ansonsten in SW viel, viel langsamer.

Beispiel 12: AES (Advanced Encryption Standard)





AES wurde in einem Wettbewerb der NIST (US National Institute of Standards and Technology) ausgewählt. Es war ziemlich überraschend, dass sich der einzige nicht US-Algorithmus durchgesetzt hat.

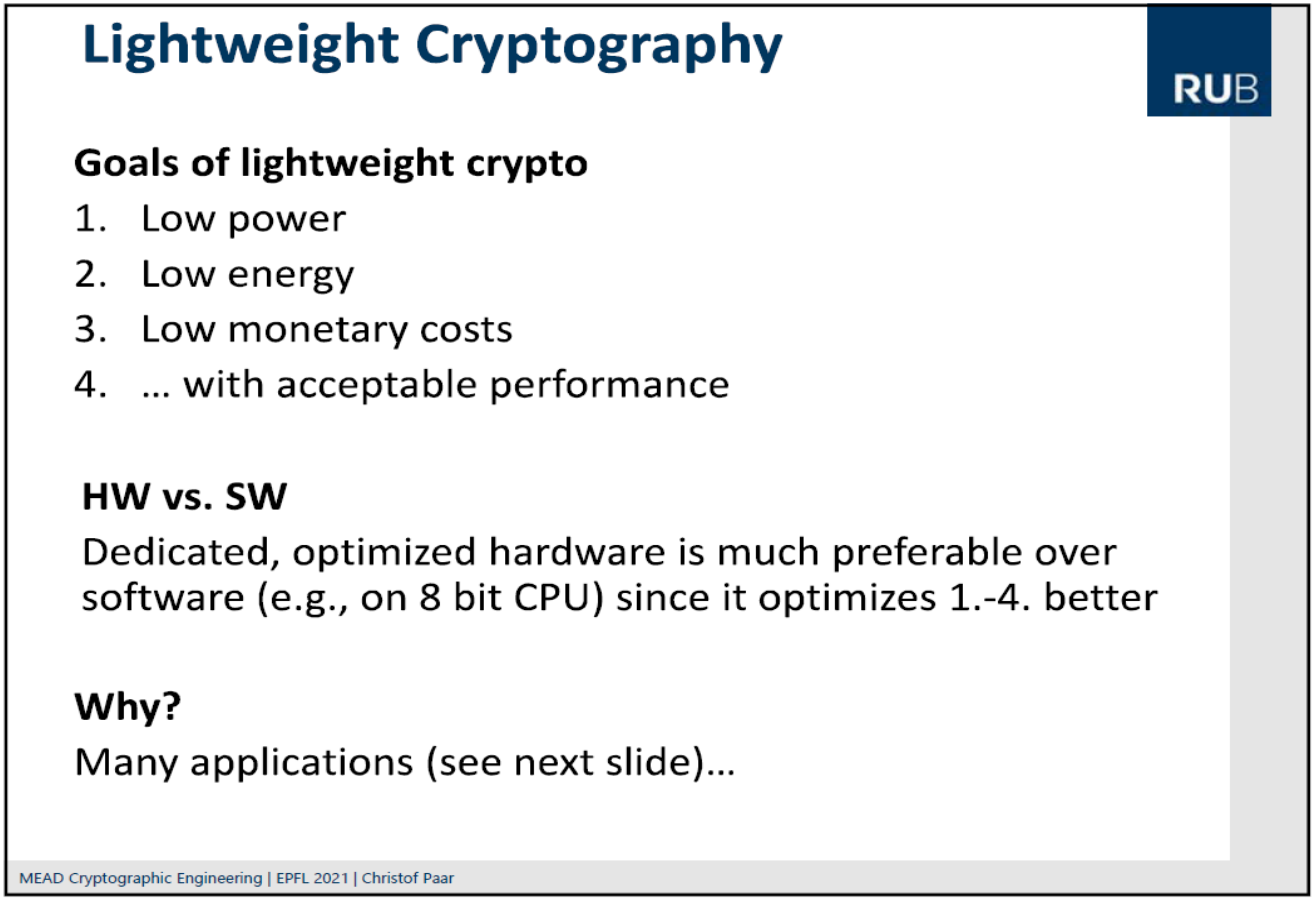
Weitere Fakts zum AES (Advanced Encryption Standard), tw. aus [CP21] zitiert:

- Seit 2000 als Nachfolger vom DES standardisiert, er wird (voraussichtlich) die nächsten 20 Jahre Bestand haben.
- Der AES ist nicht nur sicherer als der DES, sondern in HW wie als SW auch viel schneller.
- Der AES ist insbesondere auch von der NSA (National Security Agency) zugelassen, um TOP SECRET Dokumente zu verschlüsseln.
- Der AES ist ***keine*** Feistelchiffre – der Begriff ist im JS Skript in Kap. 7.2.9 erörtert, ist aber ***nicht*** Prüfungsstoff. ***Nachteil:*** „Encrypt“ und „Decrypt“ sind unterschiedlich, sie brauchen unterschiedliche Algorithmen.
- Der AES braucht für „Decrypt“ ***doppelt*** so lange wie „Encrypt“.
- Der AES mit 256 Schlüsselbits braucht nur ca. 40% länger als AES-128, dies weil 14 statt 10 Runden.
- Z.Z. sind keine relevanten Attacken bekannt, die besser als Brute-Force Attacken sind. Es gibt allerdings einen (theoretischen) Angriff, so dass der AES mit 256 Bit Schlüsselgrösse „nur“ so sicher ist, wie wenn er die Schlüsselgrösse 224 Bit hätte.
- Der AES ist „Software“-freundlich; da lernte die NIST offensichtlich etwas aus den Vorwürfen zum DES.
- Aber unterschiedliche Implementationstechniken bei Verwendung von 8-Bit Prozessoren (z.B. Chipkarten) oder 32-Bit Prozessoren (z.B. HSM). Eine SW, die für einen 32-Bit Prozessor geschrieben wurde, würde prinzipiell auf einem 8-Bit Prozessor laufen, wäre aber sehr langsam.

Einschub: Lightweight Cryptography

Warum Lightweight Cryptography?

Im Rahmen von „Internet of Things“ (IoT), Sicherung von „Gebrauchsgeständen“ wie Autos usw., Kommunikation mittels RFID und Einsatz von Low-Energy Devices ist der Wunsch gross, einen Blockchiffrierer zu haben, der eine gute Performance und einen kleinen Stromverbrauch hat und zudem wenig Platz braucht.



Einschub: Lightweight Cryptography

Applications for Lightweight Crypto

RUB

1. "We need RFID security with less than 2000 gates"
   Sanjay Sarma, AUTO-ID Labs, CHES 2002



2. Securing sensor networks, e.g., infrastructure



3. Anti-Counterfeiting ($3 trillions damage due to product piracy*)





*Source: www.bascap.com

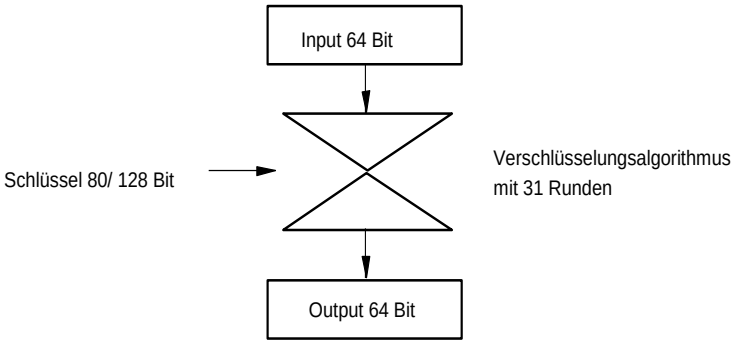
⇒ Mainly authentication & identification: can both be fixed with cryptography

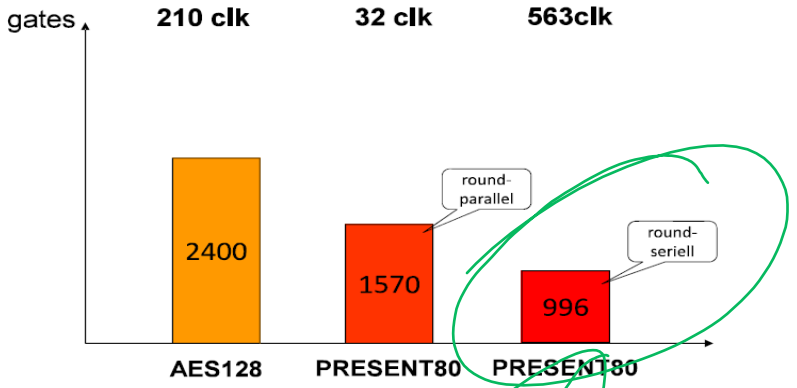
MEAD Cryptographic Engineering | EPFL 2021 | Christof Paar

Bemerkung:

Mit „trillions" ist das amerikanische Trillion = 1000 Billionen = 1000 Milliarden = 10^{12} gemeint!!
Im Deutschen ist eine Trillion = 1000 Billiarden = 1'000'000 Billionen $= 10^{18}$.
* Die oben erwähnte Zahl von „$3 trillions" entspricht in etwa der Wirtschaftsleistung von Italien.

Beispiel 13: PRESENT





- Time-area product 10 times better than smallest AES (90% energy savings)
- 128 bit version adds appr. 250 gates

Weitere Fakts zum PRESENT, tw. aus [CP21] zitiert:

- Seit 2012 standardisiert.
- Gibt es als 80 & 128 Bit Algorithmus.
- Braucht wenig Energie in HW und SW.
- SW i.d.R. für 8-Bit Prozessor optimiert.
- In optimierter HW bis 100-mal geringerer Stromverbrauch als in SW.
- Die HW für den 80 Bit Algorithmus braucht in der seriellen Implementation nur ca. 1000 Gatter. Damit kommt er nahe an die theoretische minimale Anzahl (800) von Gattern aus.
- Für den 128 Bit Algorithmus braucht es nur ca. 250 Gatter mehr.
- Es gibt viele Vorschläge für weitere ähnliche Algorithmen.
- Es gibt aktuell über 20 Kryptoanalyse Paper zu PRESENT, z.Z. sind keine relevanten Attacken bekannt, die besser als Brute-Force Attacken sind.
- Die HW für den 80 Bit Algorithmus in der parallelen Implementation braucht zwar mehr Gatter als die SW, aber die Kennzahl „Zeit · Platz“ = clk · Gatter ist ca. 10-mal kleiner als bei AES 128 (siehe Diagramm oben).

	AES	PRESENT 80, parallel	PRESENT 80, seriell
„Zeit · Platz“	210·2400 = 504’000	32·1570 = 50’240	563·996 = 560’748

Bemerkung: Im JS Skript ist mit ASCON eine weitere Lightweight Cipher aufgeführt.

Basis-Test Präsenz 2 – 3 zu Kap. 7.2

Aussage	Richtig oder falsch?	Begründung
Als Blockchiffre ist AES mit 256 Bit Schlüssellänge zu empfehlen.	☐ richtig ☐ falsch	
Ein Tripple-DES mit 3 versch. Schlüsseln hat eine Sicherheit von 112 Bit.	☐ richtig ☐ falsch	
Wird im Input einer Blockchiffre ein Bit geändert, dann ändert – bei gleichem Schlüssel – die erste Hälfte der Bits des Chiffrats.	☐ richtig ☐ falsch	
Wird ein Schlüsselbit einer Blockchiffre ein Bit geändert, dann ändern sich – bei gleichem Input – im stat. Mittel die Hälfte der Bits des Chiffrats.	☐ richtig ☐ falsch	
Das Verschlüsseln und Entschlüsseln dauern beim DES gleich lang.	☐ richtig ☐ falsch	
Das Verschlüsseln und Entschlüsseln dauern beim AES gleich lang.	☐ richtig ☐ falsch	
Beim AES-256 dauert die Verschlüsselung etwa doppelt so lange wie beim AES-128.	☐ richtig ☐ falsch	
Lightweight Cryptography wird in Zukunft immer wichtiger.	☐ richtig ☐ falsch	
Es gibt noch keine Lightweight Crypto-Algorithmen.	☐ richtig ☐ falsch	

bis 16:30
dann Pause 16:40

©Josef Schuler J∫Skripte

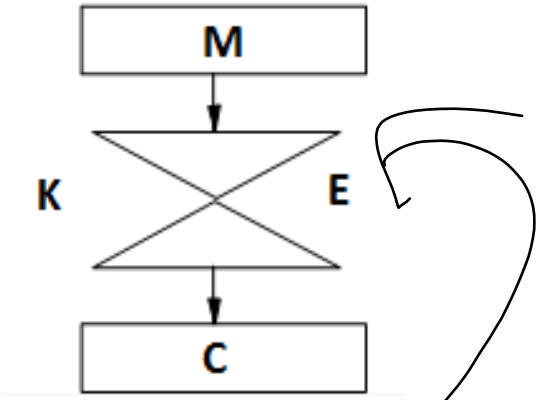
Kap. 8
TECHNIKEN BLOCKCHIFFREN

Einsatzmöglichkeiten von Blockchiffren

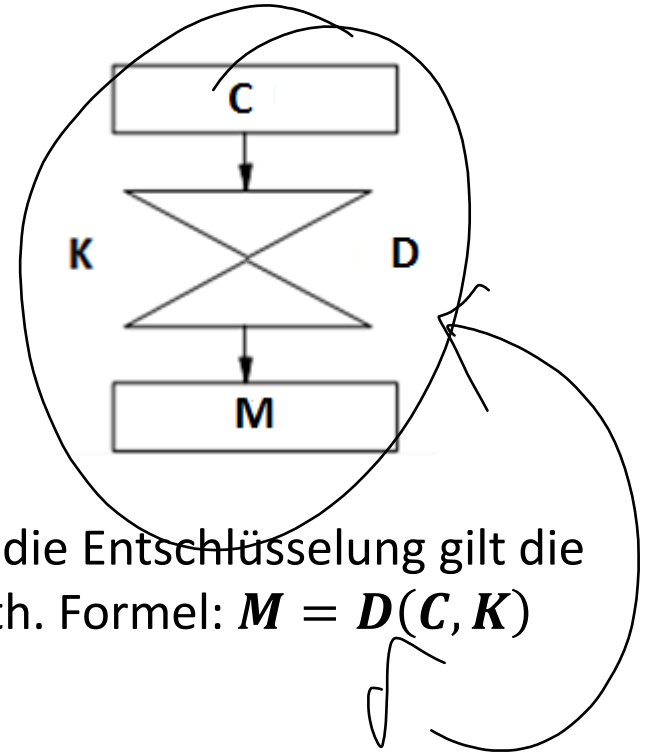
- Mit Blockchiffren kann man nicht nur – in diversen Betriebsmodi – Verschlüsseln. Nein, es gibt auch noch andere Einsatzmöglichkeiten für Blockchiffren.
  - Verschlüsselung mit Blockchiffren im Blockmodus, cf. Kap. 8.1.
  - Schlüsselgenerierung für Streamcipher, d.h. Verschlüsselung mit Blockchiffren im Streamciphermodus cf. Kap. 8.2.
  - Integritätsschutz mittels CBC-MAC Berechnung cf. Kap. 8.3.
  - Randomgenerator z.B. für Sessionkeyerzeugung, cf. Kap. 8.4.
  - Das wird auch in der Übersichtszeichnung in Folie 36 unter dem rotem Pfeil aufgelistet.
- Neben den im Folgenden behandelten Betriebsmodi gibt es noch eine ganze Reihe weiterer, z.B.
  - CFB
  - GCM
  - CCM
  - Uvm.
- Spannend sind kombinierte Modi (Verschlüsseln und Integrität gewährleisten, d.h. MAC'en) als Ersatz des Galois Counter Mode (GCM), da dieser Schwächen enthält. Mittels einem Wettbewerb wurden zwei Algorithmen eruiert: ACRON & AEGIS; wir gehen aber nicht weiter darauf ein.
- Die Idee von Kombi Modi (Verschlüsseln und Integrität gewährleisten) ist nicht absolut neu, siehe dazu weiter hinten.

Mathematische Darstellung von Blockchiffren

Im Folgenden soll eine Blockchiffre wie folgt in mathematischer Form formuliert werden:



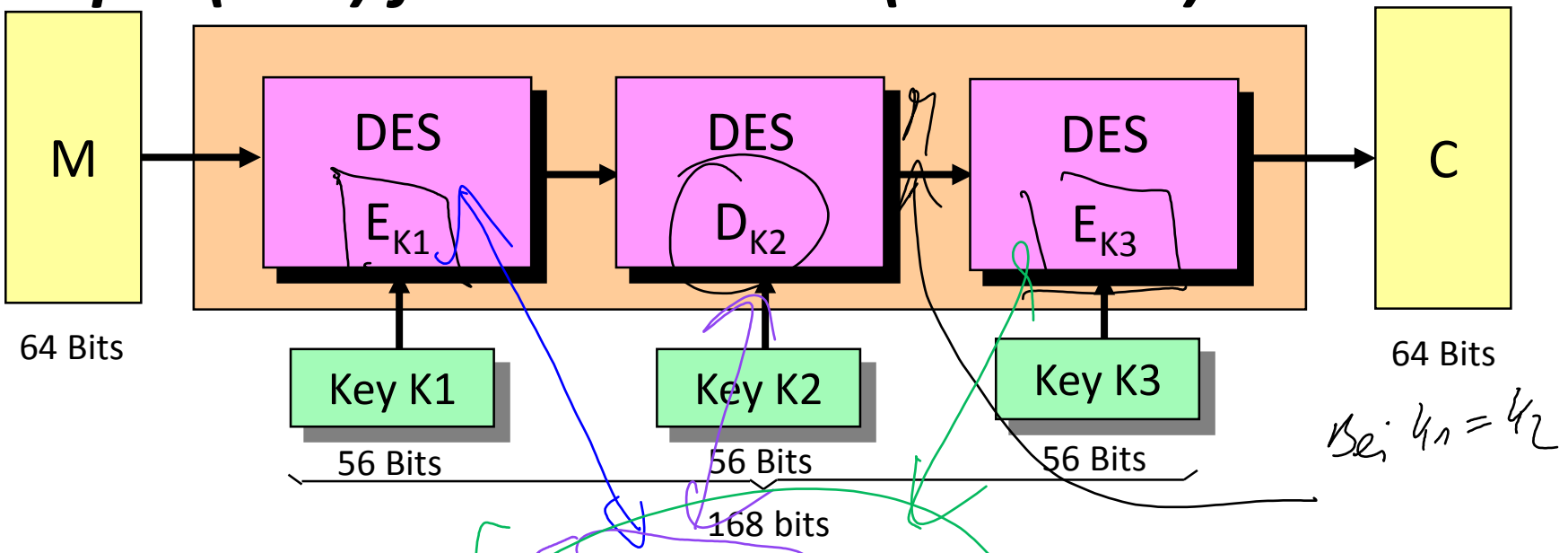
Für die Verschlüsselung gilt die math. Formel: $\boldsymbol{C = E(M, K)}$



Für die Entschlüsselung gilt die math. Formel: $\boldsymbol{M = D(C, K)}$

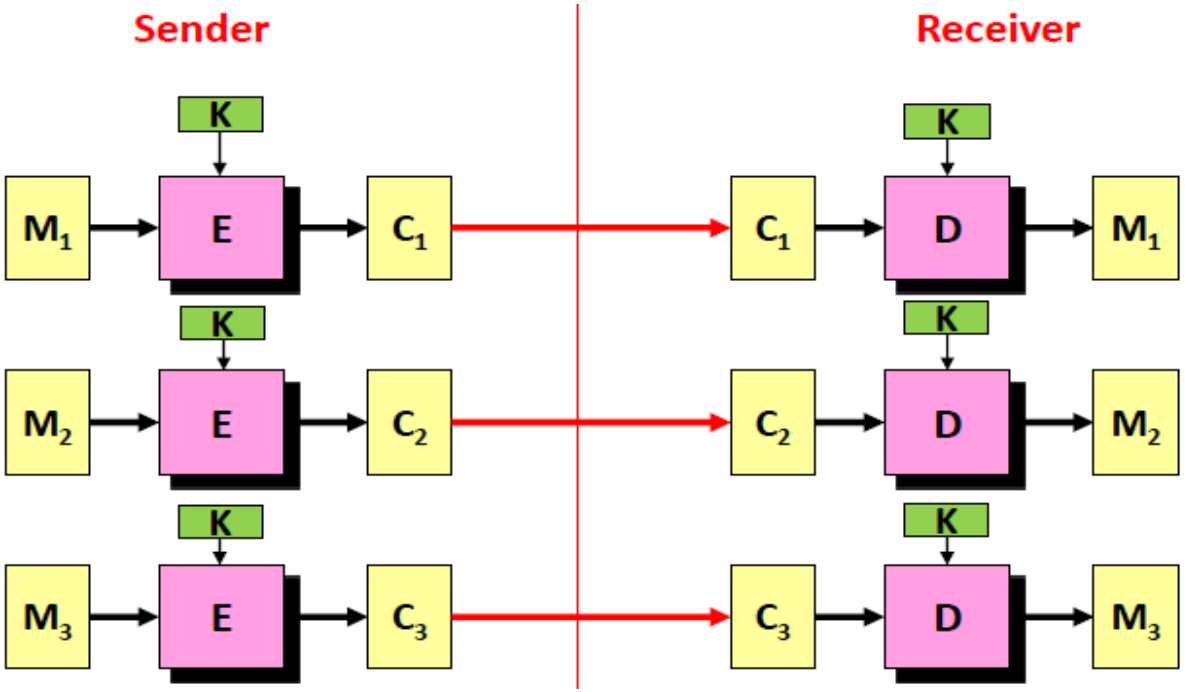
Kap. 8.1
VERSCHLÜSSELUNG MIT BLOCKCHIFFREN IM BLOCKMODUS

Triple (DES) für einen Block (ISO 8732)



- In Math. Kurzform: $\boldsymbol{C = E(D(E(M, K_1), K_2), K_3)}$.
- **Aufgabe 8.1** Schreiben Sie die Kurzform der Entschlüsselung auf.
- Neben der oben aufgeführten Variante, gibt es noch eine mit 2 Keys: K1 = K3
- Die EDE-Form wurde wegen der Rückwärtskompatibilität zum Single-DES gewählt (K1 = K2 = K3).
- Triple-AES ist z.Z. nicht nötig, da AES mit 256 Bits existiert.
- Der two-Key TDES (= 3DES) ist seit 2009 nicht mehr erlaubt. Sicherheit viel, viel kleiner als 112 Bit, nämlich nur ca. 57 – 60 Bit!
- Der three-Key TDES (= 3DES) ist gemäss NIST bis 2030 erlaubt, resp. sicher genug. Die Sicherheit ist ca. 112 Bit (und nicht etwa 168 Bit). Grund: Meet-in-the-middle Attacke (cf. Präsenz 5 „Kryptoanalyse“).

ECB-Modus für mehrere Blöcke



Klartext



Chiffretext



- ECB = Electronic Codebook
- In Math. Kurzform: $\boldsymbol{C_i = E(M_i, K)}$
- **Aufgabe 8.2** Schreiben Sie die Kurzform der Entschlüsselung auf.

\+ Die Verschlüsselung und Entschlüsselung ist parallelisierbar.

\+ Eine Teilverschlüsselung und -entschlüsselung eines grossen Files oder Harddisk ist möglich.

\+ Ein Verändern im Chiffretext wirkt sich beim Entschl. nur auf den direkt betroffenen Block aus.

— Gleiche Klartextblöcke werden in gleiche Chiffratblöcke verschlüsselt (cf. Bilder oben rechts).

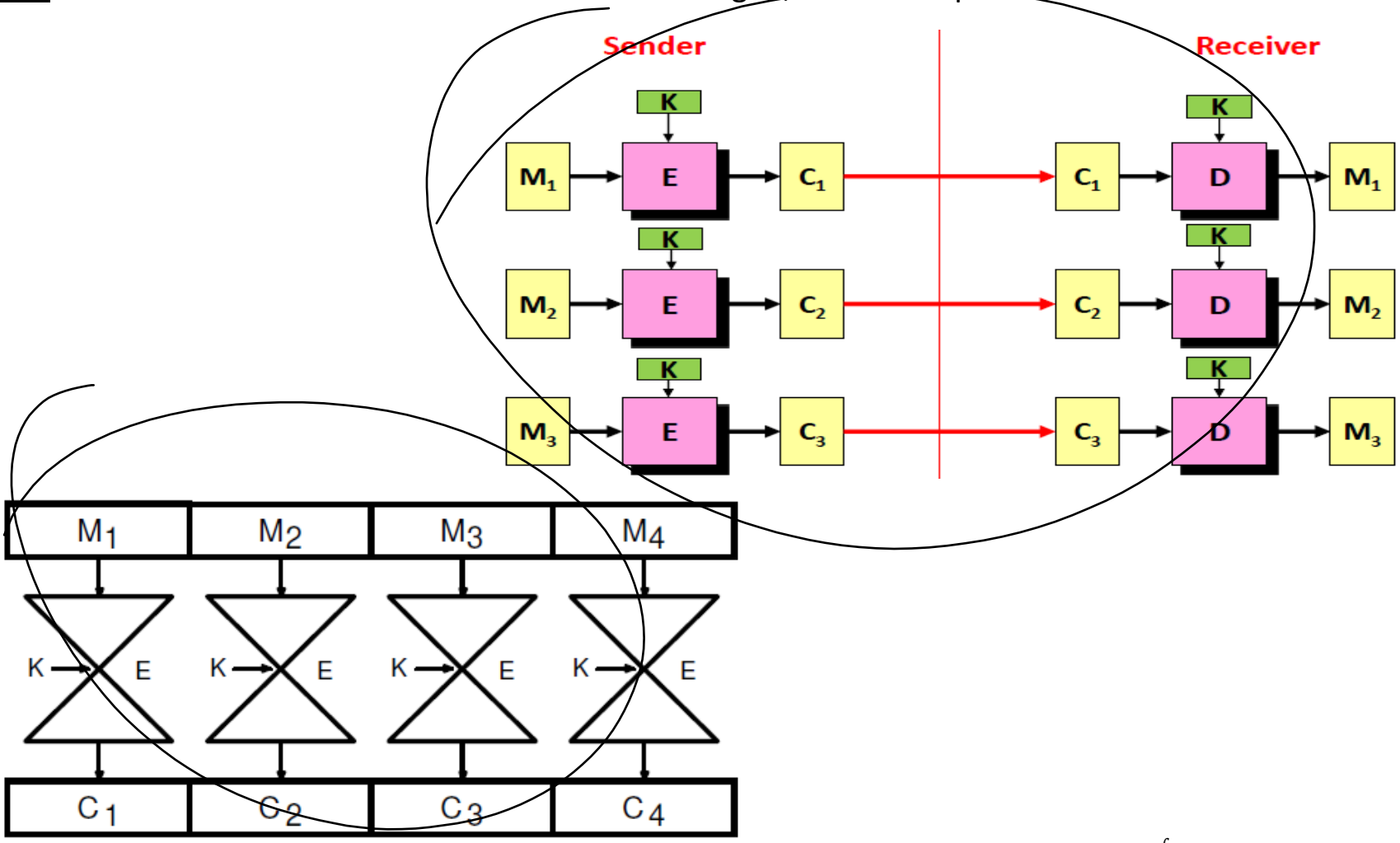
— Ein Vertauschen von Chiffratblöcken wird beim Entschlüsseln ***nicht*** bemerkt.

— Problem, wenn eine Meldungslänge nicht ein Vielfaches der Blocklänge ist. Lösung z.B. Padding oder mit Hilfe eines „Tricks“ (siehe CBC-Modus).

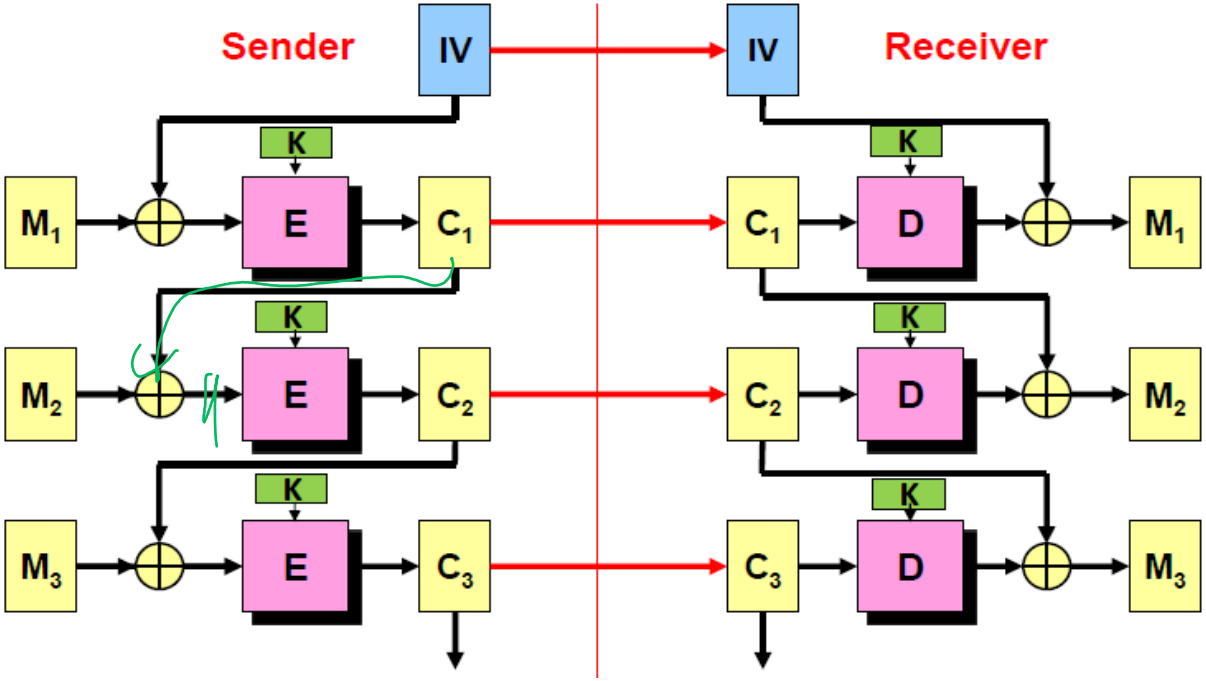
Einschub: Verschiedene Darstellungen

Sie merken sehr schnell, dass ich sowohl in den Folien und im Skript für gleiche Algorithmen verschiedene Darstellungen angebe. Das ist auch in den Büchern so: es gibt unzählige mögliche Darstellungen.

Bitte: Gewöhnen Sie sich an meine versch. Darstellungen, Sie sollten problemlos wechseln können.



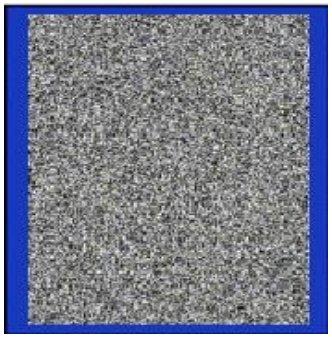
CBC-Modus, CBC = Cipher Block Chaining



Klartext



Chiffretext



- In Math. Kurzform: $C_i = E(M_i \oplus C_{i-1}, K)$, mit $C_0 = Initial\ Value$
- **Aufgabe 8.3** Schreiben Sie die Kurzform der Entschlüsselung auf.

— Die Verschlüsselung ist ***nicht*** parallelisierbar. ***Bemerkung:*** Die Entschlüsselung hingegen schon!

— Die Teilverschlüsselung und -entschlüsselung eines grossen Files oder Harddisk ist ***nicht*** möglich.

— Problem, wenn eine Meldungslänge nicht ein Vielfaches der Blocklänge ist.

\+ Ein Verändern im Chiffretext wirkt sich beim Entschl. auf den direkt betroffenen Block und in einem Bit des nächsten Blocks aus.

\+ Gleiche Klartextblöcke werden ***nicht*** in gleiche Chiffratblöcke verschlüsselt (Bilder oben rechts).

\+ Ein Vertauschen von Chiffratblöcken ist nicht mehr so einfach möglich.

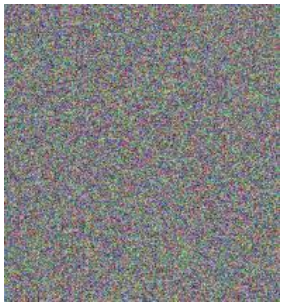
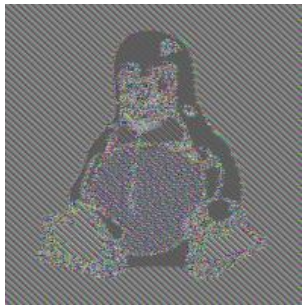
CBC-Modus, Fort.

Frage 1: Ist der Initial Value IV ein Key und für was braucht es ihn?

Antwort 1:

- Der IV = Initial Value ist kein Schlüssel und muss deshalb auch nicht geheim gehalten werden. Er kann z.B. im unverschlüsseltem Message Header mitgeschickt werden.
- Der IV dient zwei Zwecken:
  (1) Mit dem IV ist die Verschlüsselung für jeden Block softwaretechnisch gesehen gleich. Ohne IV müsste die SW checken, ob ein Block der zu verschlüsseln ist, der erste – also ohne IV – ist oder nicht.
  (2) Der wechselnde IV garantiert, dass die Chiffrate einer Meldung, die mit dem gleichen Key wiederholt verschlüsselt wird (***Achtung:*** so oder so nicht zu empfehlen) unterschiedlich werden.

Aufgabe 8.4

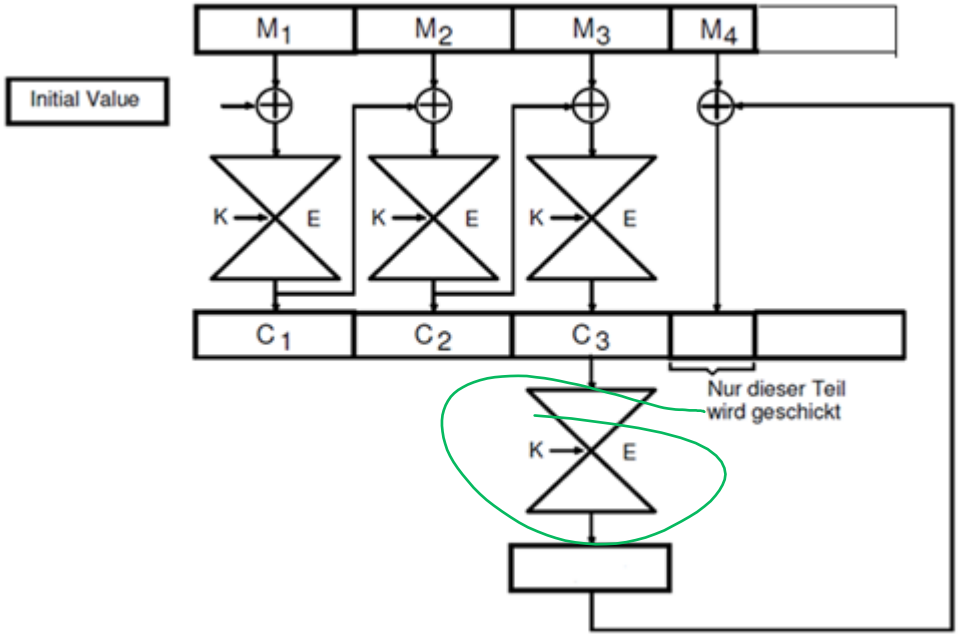
Meldung	Output Typ 1	Output Typ 2
		
Mit Modus … verschlüsselt		

CBC-Modus, Fort.

Frage 2: Was macht man, wenn die Meldungslänge kein Vielfaches der Blocklänge ist? Resp. Wie lautet nun der angepriesene „Tricks“?

Antwort 2:

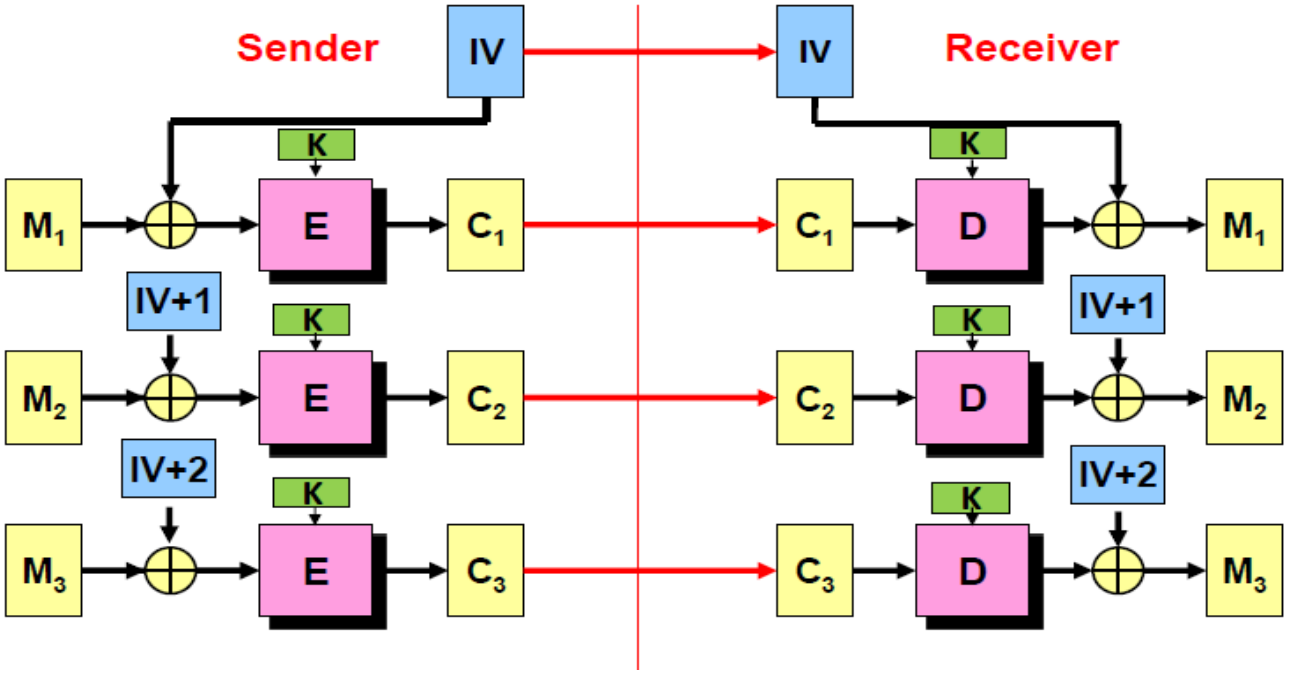
Den letzten Block speziell verschlüsseln, siehe Zeichnung.



Aufgabe 8.5 Füllen Sie die offenen Stellen des Satzes aus.

Der 4-te Block ist eine _____ mit dem nochmals verschlüsselten _____ als Schlüssel. Die Formel für den letzten Block lautet: _____. Die überflüssigen Bits werden _____. Es ist zu beachten, dass bei der Entschlüsselung die Formel für den 4-ten Block _____. D.h. auch bei der Entschlüsselung muss für die Schlüsselerzeugung eine _____ gemacht werden!

Didakt. Gründe: ein nicht publ. „Mix Modus“



Dieser nicht publizierte „Mix-Modus“ vereinigt die Vorteile von ECB und CBC.

\+ Die Verschlüsselung und Entschlüsselung ist parallelisierbar.

\+ Eine Teilverschlüsselung und -entschlüsselung eines grossen Files oder Harddisk ist möglich.

\+ Gleiche Klartextblöcke werden ***nicht*** in gleiche Chiffratblöcke verschlüsselt.

\+ Ein Vertauschen von Chiffratblöcken ist nicht mehr so einfach möglich.

Aufgabe 8.6

a) Schreiben Sie die Kurzform der Ver- und Entschlüsselung auf.

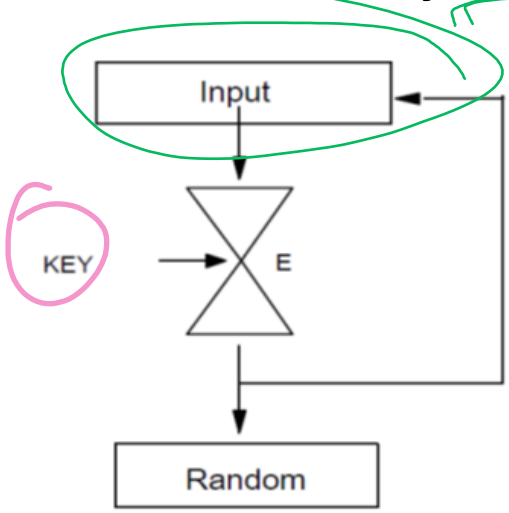
b) Im schlechtesten Fall ändert sich bei $M_1 = M_2$ nur ein Bit im Input, genügt das?

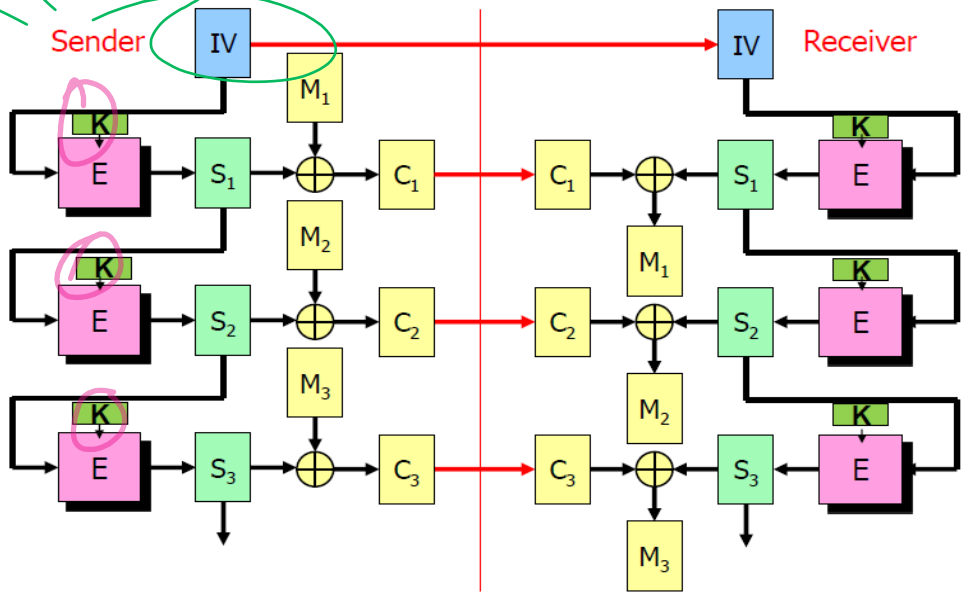
c) Was war das Initialereignis für das Designen eines Modus mit all diesen Eigenschaften?

Weiterentwicklung: Der CTR Mode (CTR = Counter Mode), cf. Kap. 8.2.

Kap. 8.2
VERSCHLÜSSELUNG MIT BLOCKCHIFFREN IM STREAMCIPHERMODUS

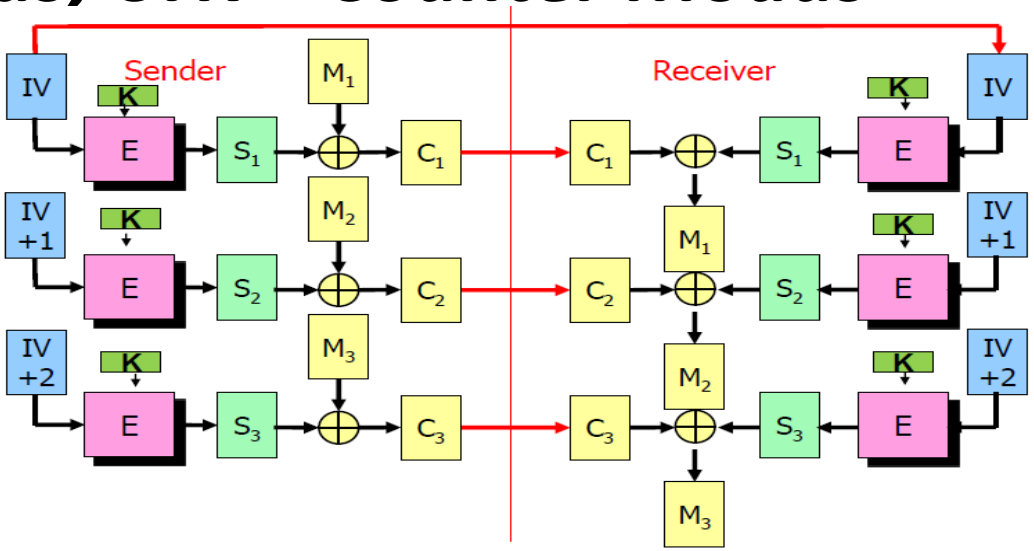
OFB-Modus, OFB = Output Feedback





- Die Idee ist nun, dass die Blockchiffre nicht für die Verschlüsselung, sondern „nur“ als Pseudo Zufallszahlengenerator, also für die Schlüsselerzeugung gebraucht wird.
- Eine Stromchiffre hat immer den Vorteil, dass die Bedingung „ein Vielfaches der Blocklänge“ nicht erfüllt sein muss.
- In Math. Kurzform: $\boldsymbol{C_i = S_i \oplus M_i}$, mit $\boldsymbol{S_i = E(S_{i-1}, K)}$ und $\boldsymbol{S_0 = Initial\ Value}$
- **Aufgabe 8.7** Schreiben Sie die Kurzform der Entschlüsselung (Decrypt) auf. Vervollständigen Sie den Satz: Auch beim Entschlüsseln ist die Blockchiffre im _____ Modus.
- Irgendeinmal wiederholen sich die Schlüssel, dies geschieht spätestens nach 2^n Durchgängen. Zweimal der gleiche Schlüssel ist bei einer Stromchiffre der Supergau.
- Auf Grund von Wahrscheinlichkeitstheoretischen Gegebenheiten besteht die mittlere Zykluslänge aus $\sqrt{2^n}$, wenn n die Blockgrösse ist. D.h. bei n = 64 ist das $\sqrt{2^{64}} = 2^{32}$, also ca. 4 Milliarden, also für heutige Verhältnisse relativ klein. Bei n = 128 ist das $\sqrt{2^{128}} = 2^{64}$.

CTR-Modus, CTR = Counter Modus



- In Anlehnung an den nicht publizierten, aber aus didaktischen Gründen behandelten „Mix Modus“ (eine Blockchiffre) erfüllt der CTR – Modus als Stromchiffre die gleichen guten Eigenschaften:
  - \+ Die Verschlüsselung und Entschlüsselung ist parallelisierbar.
  - \+ Eine Teilverschlüsselung und -entschlüsselung eines grossen Files oder Harddisk ist möglich.
  - \+ Vorausberechnung vom Schlüsselmaterial ist möglich.
  - \+ Gleiche Klartextblöcke werden ***nicht*** in gleiche Chiffratblöcke verschlüsselt.
  - \+ Ein Vertauschen von Chiffratblöcken ist nicht mehr so einfach möglich.

- In Math. Kurzform: $\boldsymbol{C_i = S_i \oplus M_i}$, mit $\boldsymbol{S_i = E(IV + (i-1), K)}$
- Auch hier gilt, dass bei der Entschlüsselung die Blockchiffre im Encrypt Modus ist.
- Wenn im schlechtesten Fall nur 1 Bit im Input geändert wird ist das kein Problem, cf. A. 8.6b).

Wie schon zu Beginn des Kap. 8 erwähnt, gibt es noch viele weitere Modi, insbesondere spannend sind die Kombi Modi, die Verschlüsseln und MAC’en in einem Aufruf machen können.

Fehlerfortpflanzung bei der Entschlüsselung

Die grosse Preisfrage:

Was passiert, wenn im Chiffrat ein einziges Bit gekippt wird?

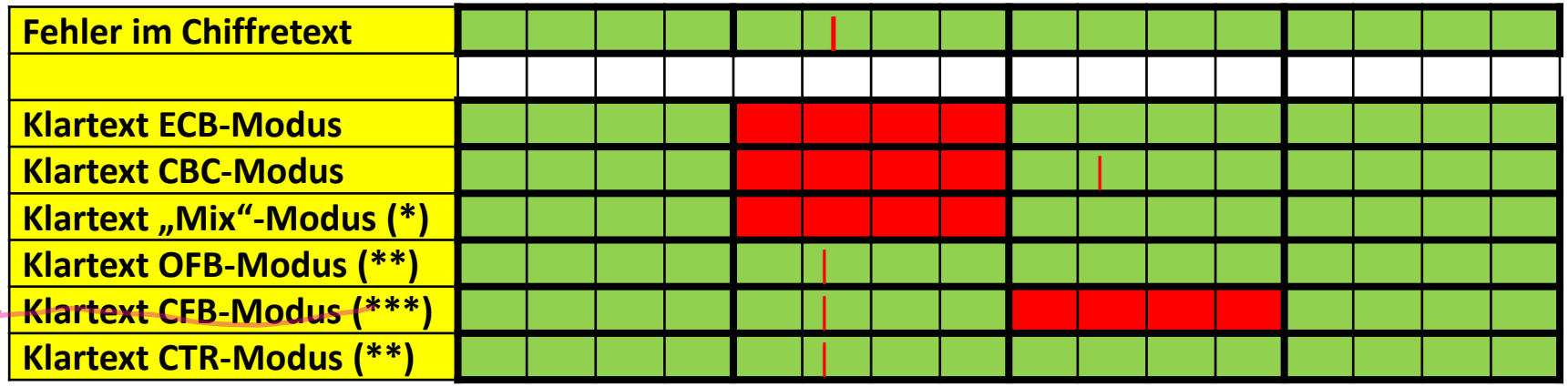
Die spontane Antwort, die i.d.R. gegeben wird:

Alles ist nun verändert, da kann nichts mehr entziffert werden!

Nur das ist grundlegend falsch:

Grün = unveränderte Bits

Dicke Umrandung = ein Block mit Blocklänge des Blockchiffriers.



(*) Kein offizieller Modus, ist aber eine Art ECB Modus.

(**) Stromchiffren

(***) Im JS Skript enthalten, aber nicht in den Folien behandelt, ***nicht*** Prüfungsstoff.

Bemerkung: Im JS Skript, Kap. 8.1.6 & 23 hat es Beispiele dazu.

Fehlerfortpflanzung b. d. Entschlüsselung, Fort.

Aufgabe 8.8
Richtig oder falsch, begründen Sie!

bis 17435

In einem Kryptomodul werden die Meldungen mit einem AES im CBC-Modus verschlüsselt. Der Integritätsschutz ist wie folgt implementiert:
Im ersten und letzten 16-Bit Block stehen die Ziffern 1234. Der Empfänger argumentiert, dass wenn nach dem entschlüsseln im erhaltenen Klartext nach wie vor im ersten und letzten Block die Ziffern 1234 stehen, dann wurde die Meldung nicht verändert.

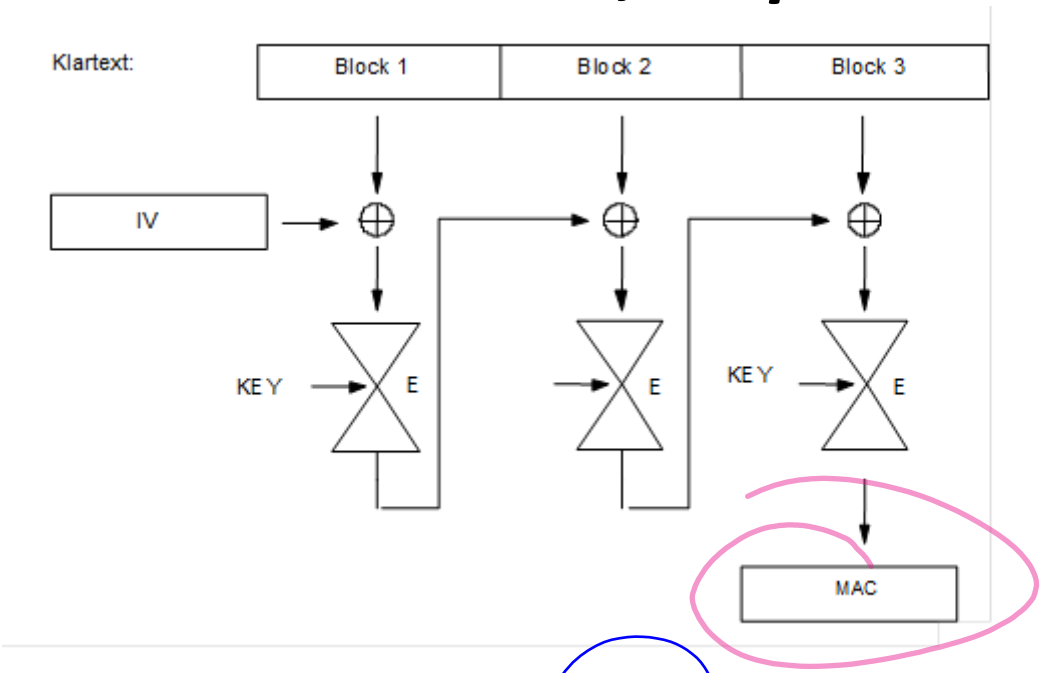


Kap. 8.3
INTEGRITÄTSSCHUTZMECHANISMEN

MIT BLOCKCHIFFREN

Dieses Kapitel wird in Präsenz 3 nur ganz kurz angesprochen. Die detaillierte Behandlung ist ein wesentlicher Teil der Präsenz 4. Die folgenden Folien sind ebenfalls in den Folien der Präsenz 4 enthalten. Insbesondere betrachten wir in Präsenz 4 auch den Standard Integritätsmechanismus, der mit Hashfunktionen gebildet wird, der sogenannte HMAC

Der CBC-MAC ANSI X9.9, resp. ISO 8730



- Der CBC-MAC, wie oben dargestellt, ist die Mutter aller Integritätsschutzmechanismen mit Blockchiffren. Er wird in allen Lehrbüchern dargestellt.
- Er hat gewisse (theoretische) Schwächen und ist eigentlich als Standard schon länger zurückgezogen.
- Es gibt diverse Anpassungen (cf. nächste Folie), gemäss ISO 9797-1 → ist aber auch als Standard zurückgezogen.
- In der Industrie gibt es viele individuelle weitere leicht angepasste Versionen.
- All diese Formen werden aber nach wie vor konkret verwendet. Dies gemäss Aussagen von Securityspezialisten, die „nahe am Geschehen sind“, so im Sinne „never change a wining team“.

Der CBC-MAC: Eine Betrachtung des IV

Grundsätzlich hätte der IV die gleiche Bedeutung wie beim CBC-Verschlüsselung Modus, d.h.

a) Ohne IV müsste die SW checken, ob ein Block der zu MACen ist, der erste ist oder nicht.

b) Die gleiche Meldung mit gleichen Key K nochmals geMaced ergibt ohne IV den gleichen MAC.

Da der IV – wie beim CBC Verschlüsselungsmodus – in klar im Header mitgeschickt wird, hat das aber beim MAC einen gewichtigen Nachteil. Es gibt nun einen Angriff:

- Eve könnte nun einen neuen Block 1neu gerieren und einfügen, ohne das Resultat des ersten XOR ändert. Dann hat sie ohne Aufwand eine zweite Meldung mit gleichem MAC erzeugt.
- Eve muss demnach die folgende Berechnung durchführen:

$$IVneu \oplus Block1neu = IV \oplus Block1 \Rightarrow IVneu = Block1neu \oplus IV \oplus Block1$$

Konsequenz und Lösungsansatz:

- Den IV einfach auf 00 … 00 setzen und nicht übertragen; Vorteil a) wird beibehalten.
- Vorteil a) beibehalten kann man auch indem ein (anderer) fixer IV definiert und ebenfalls nicht übertragen wird.
- Es gäbe aber auch die Möglichkeit – um damit auch den Vorteil b) beizubehalten – dass immer ein anderer IV erzeugt wird, dieser wie beim CBC-Verschlüsselungsmodus in klar mitschickt, aber der IV in die MAC-Berechnung (z.B. am Schluss) als Input eingeht.
- Das wird aber kaum so umgesetzt.

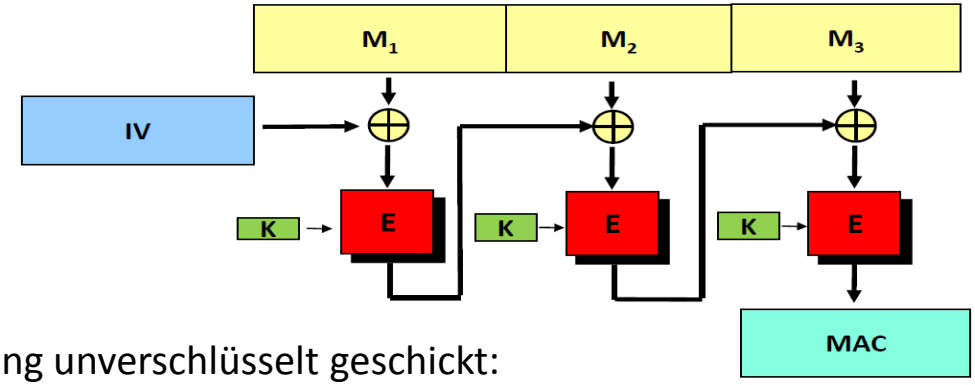
Bemerkung:

Man sieht daher in vielen Beschreibungen des CBC-MAC, dass er ohne IV da steht. Ob dann aus softwaretechnischen Gründen ein fixer 00 … 00 als IV verwendet wird, ist dann nebensächlich.

Schwäche beim CBC-MAC, eine Aufgabe

Aufgabe 8.9

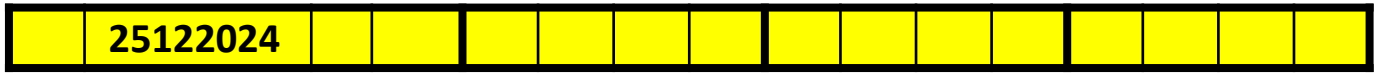
Eine Meldung wird mit dem CBC-MAC ausgestattet. Die Meldung wird unverschlüsselt und mit dem MAC versehen übermittelt. Im Folgenden sind alle Werte in HEX dargestellt, und Sie dürfen in HEX rechnen.



Folgende Meldung wird über die Leitung unverschlüsselt geschickt:



Dabei steht in Block M_1 im zweiten Feld ein Datum:



Und im IV stehen an derselben Stelle die folgenden Werte:



Der Inhalt der Meldungen M_2 und M_3 sind im Folgenden unwichtig und werden nicht angegeben. Eve kennt den Angriff auf den CBC-MAC mit dem Initial Value IV, sie fängt die ganze Nachricht ab und will das Datum 25. Dez. 2024 (= 25122024) auf den 21. Dez. 2024 ändern. Der MAC soll dabei ***unverändert*** bleiben.

Wie geht Eve vor?

Der CBC-MAC: Eine weitere Schwäche

Eine weitere Angriffsmöglichkeit beim CBC-MAC gibt es, wenn die Meldung kein Vielfaches der Blocklänge der Blockchiffre ist. In diesem Falle muss die Meldung gepaddet werden. Zunächst wurde einfach mit 00…00 gepadded.

Problem 1:

Ist das letzte Feld z.B. ein Geldbetrag, dann weiss man nicht wie gross nun der echte Betrag ist.

Lösungsansatz 1:

Das Padding wurde nun mit 10…00 gemacht. Somit wurde klar abgegrenzt, wo nun das Padding beginnt.

(Folge-)Problem 2:

Wenn die Meldung ein Vielfaches der Blocklänge des Blockchiffrieres ist, dann muss nun zusätzlich ein ganzer Paddingblock angehängt werden.

Lösungsansatz 2 (wurde leider nie standardisiert, in unseren Projekten aber implementiert):

Im Header (der Header geht auch in die MAC-Berechnung ein) der Meldung wird in einem Feld die Länge der Meldung, die in die MAC-Berechnung eingeht eingetragen.

Damit werden die obigen Angriffe unterbunden. Damit ist natürlich nicht gesagt, dass damit alle möglichen Angriffe, resp. Schwächen unterbunden werden.

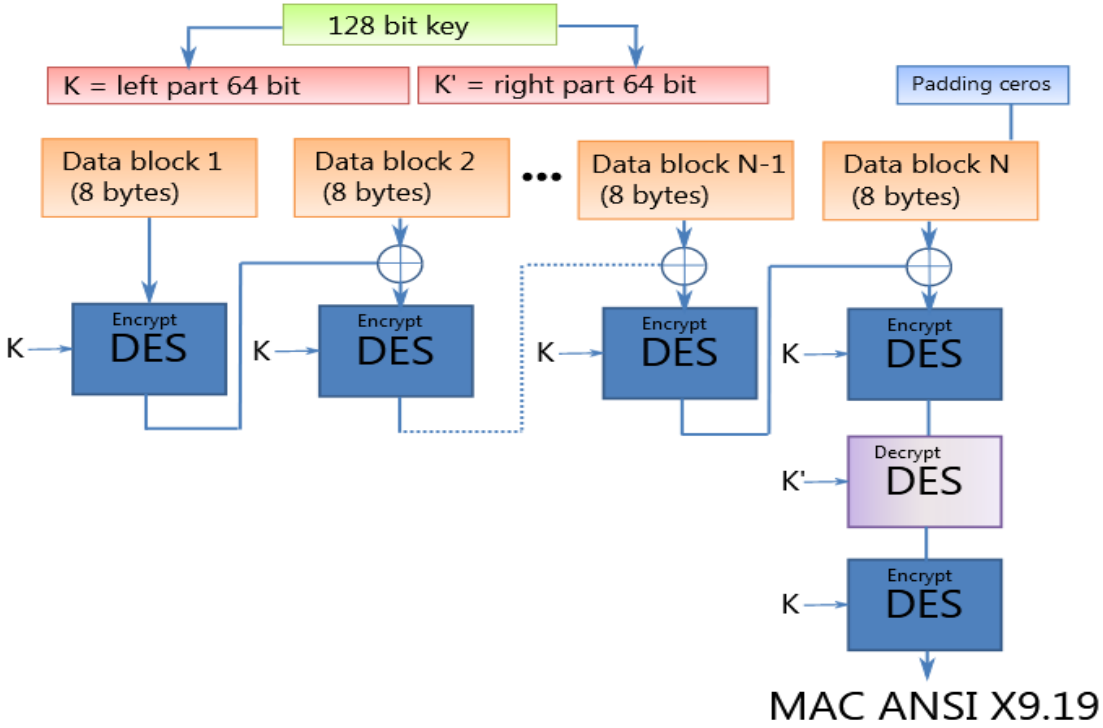
Problem 3:

Auch die nachfolgenden Verbesserungen des CBC-MAC haben die oben erwähnten Schwächen. Die Verbesserungen haben diesbezüglich keine Wirkung.

Lösungsansatz 3:

Definition von neueren MAC-Algorithmen wie CMAC usw.

Der CBC-MAC nach ISO 9797-1, resp. ISO 9807



- In ISO 9797-1 wurden 6 Varianten definiert, die oben dargestellte ist eine dieser Varianten und wurde als ANSI X9.19 Standard aufgeführt.

- Es gibt auch neuere Varianten, eben vor allem auch im Zusammenhang von kombinierten Modi.

Mathematische Beschreibung von MAC's

Analog zur Blockchiffre kann eine MAC Berechnung in einer mathematischen Form geschrieben werden. Es gibt einen IV der Länge i Bit (diese Länge kann auch 0 sein), eine beliebig grosse Meldung M der Länge * Bit („*“ bedeutet beliebige Länge) und Schlüssel K der Länge k Bit und Output MAC der Länge n Bit:

$$F: \{0,1\}^i \times \{0,1\}^* \times \{0,1\}^k \rightarrow \{0,1\}^n$$

$$IV \times M \times K \rightarrow MAC = F(IV, M, K)$$

Reihenfolge von Verschlüsseln und MAC'en

Bez. der Reihenfolge von Verschlüsseln und MAC'en, resp. Signieren ist schon länger eine relativ hitzige Debatte am Laufen, cf. JS Skript Kap. 8.3.5.
Wenn auf dem Application Layer die Kryptographie eingesetzt wird, lautet die klassische Variante ist:

<u>Über den Klartext den MAC oder die Signatur berechnen und dann Verschlüsseln.</u>

Das sieht dann in einer Übersichtszeichnung wie folgt aus:

Header	**Body oder Meldungsinhalt**	**Trailer**
Mit MAC geschützt		**MAC ist im Trailer**
unverschlüsselt	**verschlüsselt**	**unverschlüsselt**
Enthält z.B. Routingdaten, Datum, Zeit & IV	Enthält die gesamten Nutzdaten	Enthält z.B. MAC und/oder verschlüsselte Schlüssel.
Unverschlüsselt und mit MAC geschützt.	Verschlüsselt und mit MAC geschützt.	In der Regel weder noch. (**)

(**) Es gibt aber Protokolle, z.B. der Encapsulating Security Payload (ESP) bei IPSec, wo ein Teil des Trailers verschlüsselt und mit einem MAC versehen ist. In SSL 2.0 wird der MAC ebenfalls verschlüsselt der Grund liegt hier darin, dass in SSL 2.0 eine schwache MAC-Konstruktion gewählt wurde.

Authenticated Encryption, JS Skript Kap. 8.3.8 (#)

Seit der zivilen Nutzung der Kryptographie und der „Geburt“ der Integrität von G. Simmons 1970 wurden die Mechanismen der Integrität (CBC-MAC) und Verschlüsselung kombiniert. Ab ca. dem Jahre 2000 entwickelte sich daraus folgenden Themenbereiche:

- Attacken auf MAC Algorithmen und Entwerfen von entsprechenden Gegenmassnahmen.
- Diskussion um die Reihenfolge von MAC‘en und Verschlüsseln (Vor- & Nachteile sowie Attacken und Gegenmassnahmen).
- MAC’en und Verschlüsselen in einem Schritt (Kombi Modi).
- Neue Begrifflichkeiten, wobei – wie leider auch in anderen Fällen – die Definitionen nicht einheitlich sind.

Vieles davon wurde (oder wird noch) in diesem Modul angesprochen. Das Kap. 8.3.8 Authenticated Encryption soll dazu dienen bei Bedarf sich entsprechend in diese Thematik einarbeiten zu können.
Der Inhalt von Kap. 8.3.8 ist aber ***nicht*** prüfungsrelevant!!

Bemerkung:
In den Beschreibungen der Projektideen (SecProj, WiPro, BAA), Kap. 25 ist dazu eine Arbeit ausgeschrieben.

Kap. 8.4
RANDOMFUNKTIONEN

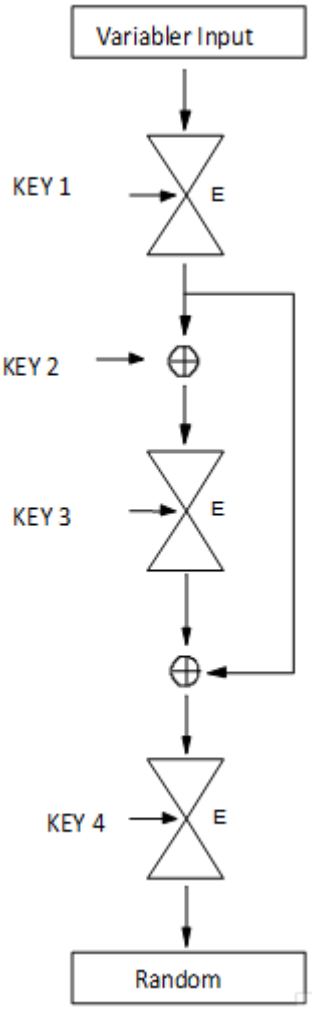
MIT BLOCKCHIFFREN

Dieses Kapitel wird in Präsenz 3 nur ganz kurz angesprochen. Die detaillierte Behandlung ist ein wesentlicher Teil der Präsenz 4. Die folgenden Folien sind ebenfalls in den Folien der Präsenz 4 enthalten. Insbesondere werden dort auch Folien gezeigt, wie man mit Hashfunktionen Zufallszahlen erzeugt.

Beispiel 14 einer Randomfkt. mit Blockchiffren

Bemerkungen:

- Die Version mit nur 2 verschiedenen Schlüsseln (Key 3 = Key 4 = Key 1) war früher im Annex von ISO 8732 standardisiert. Der Standard – und damit der Annex – wurde zurückgezogen.
- ***Nie*** Hardware Zufallszahlengeneratoren (z.B. Rauschdioden) alleine verwenden.
- Wenn ein Hardware Zufallszahlengenerator (z.B. Rauschdiode) zur Verfügung steht, dann soll er nur benutzt werden, um den obigen Input zu verbessern.
- **Vorschlag:** Input = (Variabler Input) $\oplus$ Output der „Rauschdiode“. „Variabler Input“ z.B. = Datum || Zeit
- **Analyse des Vorschlags:** Sollte die Rauschdiode kleine Zyklen liefern, ist der Input immer noch so gut, wie (Variabler Input) alleine.
- Wird eine solche Funktion zur Erzeugung von Schlüsseln gebraucht, dann nennt man sie auch ***KDF (Key Derivation Function).***
- Im Rahmen des Key Managements (cf. Fortsetzungsmodul KRYPTOE) werden wir auf diese Funktion zurückkommen.



Beispiel 15 Randomfkt. Math. dargestellt

Eine so kompliziertere Zusammensetzung wird am besten in Teilschritte aufgeteilt:

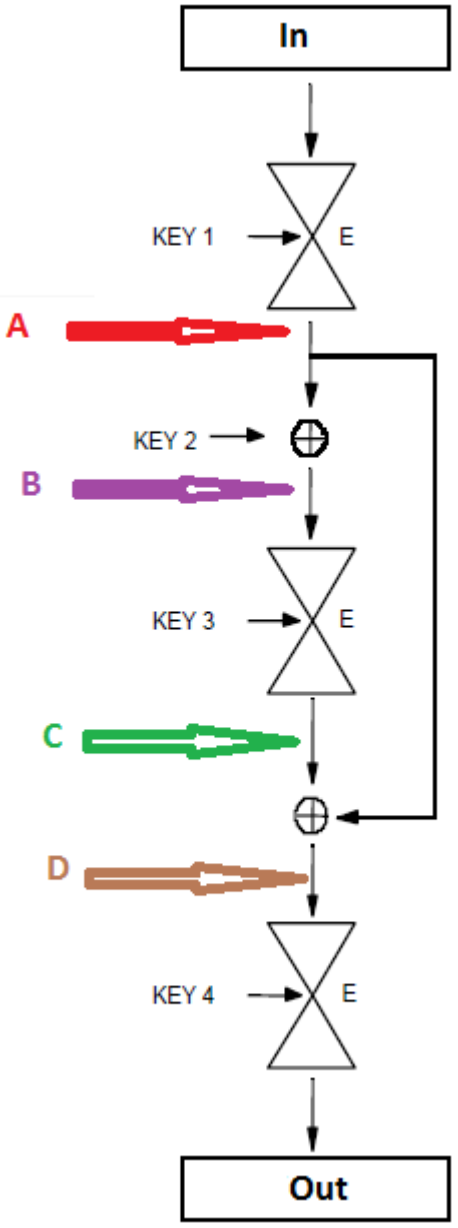
Schritt 1: $A = E(In, K_1)$

Schritt 2: $B = A \oplus K_2 = E(In, K_1) \oplus K_2$

Schritt 3: $C = E(B, K_3) = E(A \oplus K_2, K_3) = E\big((E(In, K_1) \oplus K_2), K_3\big)$

Schritt 4: $D = C \oplus A = E(A \oplus K_2, K_3) \oplus A = E\big((E(In, K_1) \oplus K_2), K_3\big) \oplus E(In, K_1)$

Schritt 5: $Out = E(D, K_4) = E(C \oplus A, K_4) = E(E(A \oplus K_2, K_3) \oplus A, K_4)$
$= E\big(E\big((E(In, K_1) \oplus K_2), K_3\big) \oplus E(In, K_1), K_4\big)$



Basis-Test Präsenz 3 zu Kap. 8 HA

Aussage	Richtig oder falsch?	Begründung
Blockchiffren haben vielfältige Einsatzmöglichkeiten.	☐ richtig ☐ falsch	
Die zwei Betriebsmodi ECB und CBC von Blockchiffrieren haben die gleichen Eigenschaften.	☐ richtig ☐ falsch	
Der CTR Modus verbindet die guten Eigenschaften von ECB und CBC.	☐ richtig ☐ falsch	
Im OFB und CTR Modus wird nur der Schlüssel mit einer Blockchiffre erzeugt. Die Verschlüsselung ist eine Stromchiffre.	☐ richtig ☐ falsch	
Wird bei einer Meldung, die mit dem CBC Modus verschlüsselt ist, ein Bit im Chiffrat gekippt, dann ist ab diesem Block die entschlüsselte Meldung völlig kaputt.	☐ richtig ☐ falsch	
Der CBC-MAC ist ein Modus, um die Meldungsintegrität zu gewähren.	☐ richtig ☐ falsch	
Der HMAC ist ein weiterer Modus, wie man mit Blockchiffren Integrität gewähren kann.	☐ richtig ☐ falsch	
Eine Triple-DES Entschlüsselung mit 3 verschiedenen Schlüsseln kann mit $\boldsymbol{M = D(D(D(C, K_1), K_2), K_3)}$ beschrieben werden. Dabei ist M der Klartext und C das Chiffrat.	☐ richtig ☐ falsch	

ZUSAMMENFASSUNG DER VORBEREITUNG (PRÄSENZ 0) UND VON DEN PRÄSENZEN 1 – 3

Zusammenfassung I

- Vertraulichkeit/Geheimhaltung
  - Massnahmen
    - symmetrische Verschlüsselung
      - Verfahren: Strom- und Blochchiffren
      - Algorithmen: DES(*); 3-DES(**), AES, IDEA usw.
      - Modi: ECB(*), CBC, OFB, CFB, CTR usw.
      - (*) „darf“ nicht mehr verwendet werden.
      - (**) „sollte“ nicht mehr verwendet werden.
    - asymmetrische Verschlüsselung → siehe Präsenzen 7 & ff.
      - Verfahren: diskrete Logarithmen, diskretes Wurzelziehen
      - Algorithmen: ECC (Elliptic Curve), RSA, Elgamal usw.
      - Modi: nichts Vergleichbares wie für symmetrische Verfahren.
  - Verschlüsselung schützt nur vor Abhören!!
- Daten-Integrität/Daten-Authentizität
  - Massnahmen:
    - MAC-Berechnung (symmetrisch)
      - Schützt vor Verfälschen und Einfügen
      - CBC-MAC (auf Blochchiffren beruhend, cf. Kap. 8.3)
        - Vor allem in „Zahlungswelt“ verwendet
      - HMAC (auf Hashfunktionen beruhend) → siehe Präsenz 4.
        - Vor allem in der „Netzwerkwelt“ verbreitet.

Zusammenfassung II

- Daten-Integrität/Daten-Authentizität, Fortsetzung
  - Massnahmen, Fortsetzung:
    - Digitale Signatur (asymmetrisch) → siehe Präsenzen 7 & ff.
      - Schützt vor Verfälschen, Einfügen und Abstreiten eine Meldung geschickt zu haben.
      - Verfahren: diskrete Logarithmen, diskretes Wurzelziehen
      - Algorithmen: ECC (Elliptic Curve), RSA, DSA, Schnorr usw.
- Replay und Delete
  - Massnahmen:
    - Führen einer Sequenznummer und MAC-Berechnung (symmetrisch)
      - Schützt zusätzlich vor Replay und Delete
    - Führen einer Sequenznummer und digitale Signatur (asymmetrisch) → wird in dieser Form aber eher nicht gemacht.
      - Schützt ebenfalls zusätzlich vor Replay und Delete
- Non rep. of Receipt und Masquerade (Benutzer- resp. Instanzauthentizität, Authentisierung)
  - Massnahmen:
    - Krypt. Mechanismus (MAC oder digitale Signatur) in ein Protokoll eingebettet.

EINFÜHRUNG INS WISSENSCHAFTLICHE ARBEITEN IN DER KRYPTOLOGIE (*)
NICHT PRÜFUNGSSTOFF

Einleitung

Das wissenschaftliche Arbeiten – z.B. Schreiben einer Semesterarbeit (SecProj oder WiPro), der BSc Abschlussarbeit (BAA), später die Masterarbeit usw. – ist schlussendlich das Ziel eines Hochschulstudiums. Das wissenschaftliche Arbeiten unterscheidet sich gänzlich vom Lernen eines Modulinhalts.

- Die Aufgabenstellung ist meistens sehr offen gestellt. → Thesis, Annahme, (vage) Vermutung.
- Die Aufgabenstellung kann sich aufgrund erster Erkenntnisse ändern.
- Das Ergebnis kann grundsätzlich offen sein (z.B. es kann auch sein, dass man mit der Vermutung falsch liegt und schlussendlich zeigt, dass die Vermutung nicht stimmt).
- Es gibt nicht mehr „löse ein paar Aufgaben, in der Prüfung kommt dann eine ähnliche, dann bestehst du die Prüfung."
- ***Und insbesondere rechnet Ihnen kein Dozent die Lösung vor, es gibt auch keine vorgefertigte Musterlösung.***

Nur das wissenschaftliche Arbeiten – und erst recht dasjenige in Kryptologie – muss auch gelernt und geübt werden. Das möchte ich an dieser Stelle von mehr oder weniger offenen Aaufgabe in Kryptologie tun.
Sie sollen also ein von mir gestelltes Problem selbstständig analysieren, eine Lösungsstrategie entwickeln, ggf. weitere Informationen recherchieren und schlussendlich ein Resultat (inkl. Herleitung und Beschreibung) präsentieren.

Die Aufgabe ist ***freiwillig,*** man kann mich immer für Fragen, Zwischenkontrollen oder einer Schlussbesprechung kontaktieren.

Josef Schuler, Dozent für Mathematik und Kryptologie, josef.schuler@hslu.ch

Artikel zu „No Key Verschlüsselung“

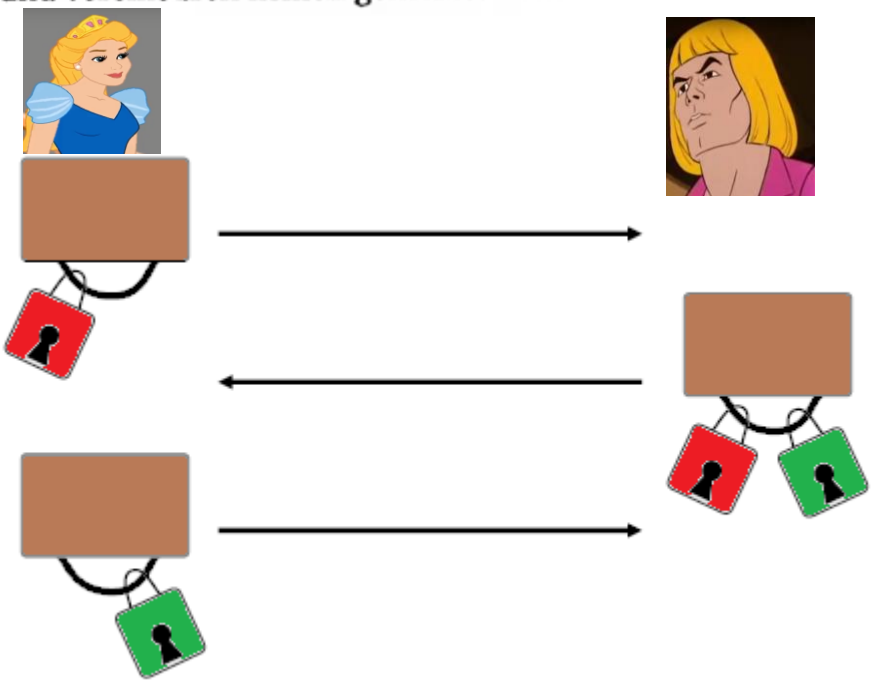
Eine Nachricht geheim über eine öffentliche Leitung verschicken, ohne dass Sender und Empfänger einen Schlüssel vereinbart haben:

Sender und Empfänger schicken die Nachricht dreimal hin und her. Auf ihrem ersten Weg vom Sender zum Empfänger ist die Nachricht mit dem geheimen Schlüssel des Senders verschlüsselt. Auf dem Rückweg ist sie doppelt verschlüsselt – zusätzlich mit dem geheimen Schlüssel des Empfängers. Auf dem erneuten Weg zum Empfänger ist sie nur noch einfach verschlüsselt – der Sender hat seine Verschlüsselung rückgängig gemacht. Wenn der Empfänger die Nachricht nun erhält, macht er seinerseits seine Verschlüsselung rückgängig und hat den Klartext vorliegen.

Auf ihrem Weg hin und her ist die Nachricht stets verschlüsselt, sodass kein Dritter sie lesen kann. Die Bezeichnung »No Key« ist insofern nicht ganz richtig, aber Sender und Empfänger verwenden nur ihre jeweils eigenen Schlüssel und vereinbaren keinen gemeinsamen Schlüssel.

Umsetzung mit Koffer, Vorhängeschlösser VHS und Kurier:

- Alice hängt rotes VHS an.
- Bob hängt grünes VHS an.
- Alice nimmt rotes VHS weg.
- Bob nimmt grünes VHS weg und kann den Koffer öffnen.



Detaillierte Aufgabenstellung

(1) Zum allgemeinen Verschlüsselungsprotokoll:
   i. Zeichnen Sie das allgemeine Verschlüsselungsprotokoll auf.
   ii. Formulieren Sie die allgemeinen Bedingungen, so dass das Protokoll funktioniert.

(2) Nun verwenden Sie a) eine Blockchiffre resp. b) eine Stromchiffre und kommentieren Sie das Protokoll, bezüglich Machbarkeit. Wenn es machbar ist, dann analysieren Sie es, ob es auch sicher ist.

a) Bei Verwendung einer Blockchiffren
b) Bei Verwendung der Stromchiffren

Tipp: Verwenden Sie die folgenden Bezeichnungen für das allgemeine Verschlüsselungsprotokoll, resp. für die Blockchiffre a).

- Verschlüsselung der Meldung M mit Schlüssel K_i ergibt Chiffrat C: $C_i = E(M; K_i)$. Dabei ist M entweder die ursprüngliche Meldung M oder dann ein Chriffrat C_i, d.h. $C_j = E(C_i; K_j)$.
- Somit ist dann eine Doppelverschlüsselung z.B. $C_2 = E(C_1; K_2) = E(E(M; K_1); K_2)$

- Entschlüsselung des Chiffrats C_i mit Schlüssel K_i ergibt Meldung M: $M = D(C_i; K_i)$. Dabei ist M entweder die ursprüngliche Meldung M oder dann ein Chriffrat C_i.
- Das Verschlüsseln und dann das Entschlüsseln muss kombiniert werden.
- Bei der Stromchiffre b) kann direkt mit $\oplus$ gearbeitet werden.
- Neben den obigen Bezeichnungen, benutzen Sie – insbesondere für (1) i. – die Vorlage auf der nächsten Folie.

Vorlage Protokollablauf

Alice mit K_1	unsichere Leitung	Bob K_2
	---------->	
	<----------	
	---------->	

©Josef Schuler $J\int$ *Skripte*

LÖSUNGEN DER AUFGABEN

Aufgabe 19.1 CE

Aufgabe 19.2 Es gibt immer Null.

Aufgabe 28.1

Ja, klar gibt es einen Haken. Das Problem ist, den Schlüssel k dem Empfänger „unendlich“ sicher zu übermitteln. Wenn ich das ja könnte, dann könnte ich ihm ja auch den zu schickenden Buchstaben (ohne zu verschlüsseln) „unendlich“ sicher zusenden.

Aufgabe 7.1 Keine Musterlösung, da es ein Arbeitsauftrag ist.

Aufgabe 7.2

a) Key3 ist das XOR von Key1 und Key2, also: $Key3 = Key1 \oplus Key2 = 8E \oplus 58 = D6$

b) Direkt gerechnet: $\text{Chiffrat} = 32 = Text \oplus Key3 \Rightarrow Text = 32 \oplus Key3 = 32 \oplus D6 = E4$
Oder: $32 = Text \oplus \left(\underbrace{Key1 \oplus Key2}_{Key3}\right) \Rightarrow Text = 32 \oplus Key3 = 32 \oplus D6 = E4$

Aufgabe 7.3

a) Schutz gegen das Abhören der Meldung.

b) Stromchiffre ist hauptsächlich ein Schutz gegen intelligente Gegner, wie z.B. Hacker, Geheimdienste usw., die einen passiven Angriff durchführen. **Grund:** Abhören ist ein passiver Angriff und in diesem Sinne sind die Angreifer passive Angreifer.

c) Symmetrische Verschlüsselung.

Aufgabe 7.4 a) Offensichtlich ging sowohl beim Sender wie Empfänger etwas schief.
b) ***Empfänger:***
So wie es aussieht ist beim Empfänger $\boldsymbol{M_1 \oplus M_2}$ als Resultat herausgekommen. Das Resultat ist – möglicherweise – aus der Berechnung von $\boldsymbol{C_1 \oplus C_2}$ entstanden.

Sender:
Beim Sender wurde (sehr wahrscheinlich) mit dem gleichen Schlüssel verschlüsselt, also z.B.
$\boldsymbol{C_1 = M_1 \oplus K_1}$ **und** $\boldsymbol{C_2 = M_2 \oplus K_1}$
Oder
$\boldsymbol{C_1 = M_1 \oplus K_2}$ **und** $\boldsymbol{C_2 = M_2 \oplus K_2}$

Mit diesen Ausgangslagen kann man nun die Puzzleteile zusammenstellen:

$$\boldsymbol{C_1 \oplus C_2 = (M_1 \oplus K_1) \oplus (M_2 \oplus K_1) = (M_1 \oplus M_2) \oplus (K_1 \oplus K_1) = M_1 \oplus M_2}$$

Oder

$$\boldsymbol{C_1 \oplus C_2 = (M_1 \oplus K_2) \oplus (M_2 \oplus K_2) = (M_1 \oplus M_2) \oplus (K_2 \oplus K_2) = M_1 \oplus M_2}$$

OK, das sind Vermutungen, nur eine genaue Analyse des Programmiercodes kann die def. Antwort liefern. Trotzdem können wir uns fragen: Wie konnte das passieren?

Sender: Z.B. Programmierfehler; es wurde in einem Falle auf einen falschen Schlüsselplatz zugegriffen.
Oder
Man hatte in beide Schlüsselplätze den gleichen Schlüssel $\boldsymbol{K_1}$ oder $\boldsymbol{K_2}$ anstatt $\boldsymbol{K_1}$ und $\boldsymbol{K_2}$ geladen, d.h. man hatte in der Schlüsseldatei $\boldsymbol{K_1||K_1}$ oder $\boldsymbol{K_2||K_2}$ anstatt $\boldsymbol{K_1||K_2}$.

Empfänger: Ein Programmierfehler ist hier die wahrscheinlichste Ursache.

Schlussbemerkung:
Es ist natürlich relativ unwahrscheinlich, dass bei Sender und Empfänger gleichzeitig so schwere Fehler passieren, aber unmöglich ist es nicht.

Aufgabe 7.5

- $2^8 = 256$ mögliche Inputs und Outputs, d.h. $M = C = \{00, 01, \dots, FF\}$.
- $2^{32} = 4'294'967'296$ mögliche Schlüssel, d.h. $K = \{00000000, 00000001, \dots, FFFFFFF\}$.
- Für jeden der $2^{32} = 4'294'967'296 \approx 4{,}3 \cdot 10^9$ Schlüssel werden also die 256 Elemente des Inputs auf die 256 Elemente des Outputs abgebildet. Also für jeden der 2^{32} Schlüssel gibt es eine Permutation von 256 Elementen, also 2^{32} Permutationen.
- Es gibt aber $256! \approx 8{,}6 \cdot 10^{506}$ Möglichkeiten 256 Elemente auf 256 Plätze zu verteilen.
- $\frac{2^{32}}{(2^8)!} = \frac{2^{32}}{256!} \approx \frac{4{,}3 \cdot 10^9}{8{,}6 \cdot 10^{509}} \approx \frac{1}{2 \cdot 10^{500}} \approx 0{,}5 \cdot 10^{-500} = 2 \cdot 10^{-499}$.
- Selbst wenn ein 256 Bit Schlüssel genommen wird, würde nur ein winziger Bruchteil der möglichen Permutationen adressiert, nämlich $\frac{2^{256}}{(2^8)!} = \frac{2^{256}}{256!} \approx \frac{1{,}15 \cdot 10^{77}}{8{,}6 \cdot 10^{509}} \approx 10^{-430}$.

Aufgabe 7.6 ***Grosser Schlüsselraum*** $\left\{\begin{matrix} \Leftarrow \\ \not\Rightarrow \\ \not\Leftrightarrow \end{matrix}\right\}$ ***Die Blockchiffre ist sicher***

Aufgabe 7.7 ***Striktes Avalanche-Kriterium*** $\left\{\begin{matrix} \Rightarrow \\ \not\Leftarrow \\ \not\Leftrightarrow \end{matrix}\right\}$ ***Avalanche-Effekt***

Aufgabe 8.1 $\boldsymbol{M = D(E(D(C, K_3), K_2), K_1)}$.

Aufgabe 8.2 $\boldsymbol{M_i = D(C_i, K)}$

Aufgabe 8.3 $\boldsymbol{M_i = D(C_i, K) \oplus C_{i-1}}$, mit $\boldsymbol{C_0 = Initial\,Value = IV}$

Aufgabe 8.4

Meldung	Output Typ 1	Output Typ 2
Mit Modus … verschlüsselt	CBC (siehe später CTR, OFB, CFB, „Mix“-Modus)	ECB

Aufgabe 8.5
Der 4-te Block ist eine Stromchiffre mit dem nochmals verschlüsselten Chiffrat C_3 als Schlüssel. Die Formel für den letzten Block lautet: $\boldsymbol{C_4 = E(C_3, K) \oplus M_4}$. Die überflüssigen Bits werden einfach abgeschnitten.
Es ist zu beachten, dass bei der Entschlüsselung die Formel für den 4-ten Block ***nicht*** ändert. D.h. auch bei der Entschlüsselung muss für die Schlüsselerzeugung eine ***Verschlüsselung*** gemacht werden!

Aufgabe 8.6

a) $\boldsymbol{C_i = E(M_i \oplus (IV + i - 1), K)}$; $\boldsymbol{M_i = D(C_i, K) \oplus (IV + i - 1)}$

b) Ja, das genügt. Der Grund liefert das strikte Avalanche Kriterium

c) Der CTR-Modus wurde in den 90-er Jahre im Zuge der Verbreitung von Laptops entwickelt. Diese wurden gestohlen oder gingen verloren und damit auch sensible Daten. Also war der Wunsch nach einer Harddiskverschlüsselung gross. Doch bei jedem noch so kleinen Zugriff auf die Daten oder noch so kleinen Änderung immer die ganze HD zu ent- und verschlüsseln war zu mühsam.

Aufgabe 8.7 $\boldsymbol{M_i = S_i \oplus C_i}$, mit $\boldsymbol{S_i = E(S_{i-1}, K)}$ und $\boldsymbol{S_0 = Initial\ Value}$
Auch beim Entschlüsseln ist die Blockchiffre im Encrypt Modus.

Aufgabe 8.8 Falsch, denn die CBC Entschlüsselung erholt sich im übernächsten Block.

Aufgabe 8.9

- Der neue Wert in $\boldsymbol{M_1}$ lautet nun: 21122024
- Nachdem Eve den Block $\boldsymbol{M_1}$ abgeändert hat, muss sie noch den $\boldsymbol{IV}$ anpassen.
- Dazu braucht sie die folgende Grundgleichung:

$$IVneu \oplus Block1neu = IV \oplus Block1 \Rightarrow IVneu = Block1neu \oplus IV \oplus Block1$$

Oder: $?\oplus Block1neu = IV \oplus Block1 = Block1neu \oplus ? \Rightarrow ? = Block1neu \oplus IV \oplus Block1$

- Sie ändert im Datum die zweite Ziffer von 5 auf 1, somit gilt an dieser Stelle

$$IVneu = Block1neu \oplus IV \oplus Block1 = 1 \oplus B \oplus 5 = A \oplus 5 = F$$

Resultat: Somit lautet der neue IV, der mitgeschickt wird: … **AFCD9876**…

Basis-Test Präsenz 2 – 3 zu Kap. 7.1

Aussage	Richtig oder falsch?	Begründung
Das wirkliche „Arbeitspferd“ der Kryptologie sind die Stromchiffren.	☐ richtig ☑ falsch	Nein, das gilt für die Blockchifffren.
Bei Stromchiffren ist die Verschlüsselung und die Entschlüsselung exakt die gleiche Operation.	☑ richtig ☐ falsch	In beiden Fällen das XOR.
Es gibt mehrere Stromchiffre Verschlüsselungsalgorithmen.	☐ richtig ☑ falsch	Es gibt aber unzählige Möglichkeiten den Schlüsselstrom zu erzeugen.
Stromchiffre und One-Time-Pad sind identische Begriffe.	☐ richtig ☑ falsch	Ein OTP ist eine Stromchiffre, aber nicht umgekehrt.
Mit Stromchiffren kann ich neben dem Verschlüsseln auch die Integrität einer Meldung gewähren.	☐ richtig ☑ falsch	Stromchiffren bieten keine Integrität.
$12 \oplus 36 = 48$	☐ richtig ☑ falsch	$12 \oplus 36 = 24$

Basis-Test Präsenz 2 – 3 zu Kap. 7.2

Aussage	Richtig oder falsch?	Begründung
Als Blockchiffre ist AES mit 256 Bit Schlüssellänge zu empfehlen.	☑ richtig ☐ falsch	Es gibt zwar noch andere Blockchiffren, aber der AES – wenn möglich mit 256 Bit – sind vorzuziehen.
Ein Tripple-DES mit 3 versch. Schlüsseln hat eine Sicherheit von 112 Bit.	☑ richtig ☐ falsch	Dies wegen einer sog. Meet-in-the-middle Attacke.
Wird im Input einer Blockchiffre ein Bit geändert, dann ändert – bei gleichem Schlüssel – die erste Hälfte der Bits des Chiffrats.	☐ richtig ☑ falsch	Es ändern im stat. Mittel die Hälfte aller Bits.
Wird ein Schlüsselbit einer Blockchiffre ein Bit geändert, dann ändern sich – bei gleichem Input – im stat. Mittel die Hälfte der Bits des Chiffrats.	☑ richtig ☐ falsch	
Das Verschlüsseln und Entschlüsseln dauern beim DES gleich lang.	☑ richtig ☐ falsch	
Das Verschlüsseln und Entschlüsseln dauern beim AES gleich lang.	☐ richtig ☑ falsch	Das Entschlüsseln braucht beim AES etwa doppelt so lange wie das Verschlüsseln.
Beim AES-256 dauert die Verschlüsselung etwa doppelt so lange wie beim AES-128.	☐ richtig ☑ falsch	Es braucht nur ca. 40% mehr Zeit. Dies weil statt 10 neu 14 Runden durchgeführt werden.
Lightweight Cryptography wird in Zukunft immer wichtiger.	☑ richtig ☐ falsch	Vornehmlich Internet der Dinge IoT.
Es gibt noch keine Lightweight Crypto-Algorithmen.	☐ richtig ☑ falsch	PRESENT ist eine Lightweight Blockchiffre.

Basis-Test Präsenz 3 zu Kap. 8

Aussage	Richtig oder falsch?	Begründung
Blockchiffren haben vielfältige Einsatzmöglichkeiten.	☑ richtig ☐ falsch	Es gibt – grob gesagt – 4 unterschiedliche Einsatzmöglichkeiten für Blockchiffren.
Die zwei Betriebsmodi ECB und CBC von Blockchiffrieren haben die gleichen Eigenschaften.	☐ richtig ☑ falsch	
Der CTR Modus verbindet die guten Eigenschaften von ECB und CBC.	☑ richtig ☐ falsch	
Im OFB und CTR Modus wird nur der Schlüssel mit einer Blockchiffre erzeugt. Die Verschlüsselung ist eine Stromchiffre.	☑ richtig ☐ falsch	
Wird bei einer Meldung, die mit dem CBC Modus verschlüsselt ist, ein Bit im Chiffrat gekippt, dann ist ab diesem Block die entschlüsselte Meldung völlig kaputt.	☐ richtig ☑ falsch	Der CBC erholt sich wieder bei der Entschlüsselung.
Der CBC-MAC ist ein Modus, um die Meldungsintegrität zu gewähren.	☑ richtig ☐ falsch	
Der HMAC ist ein weiterer Modus, wie man mit Blockchiffren Integrität gewähren kann.	☐ richtig ☑ falsch	Der HMAC und CBC-MAC (und ähnliche) verhindern die gleichen Angriffe (Integrity & Insertion). Der HMAC ist eine MAC Konstruktion mit einer Hashfunktion wie SHA-3 und ***nicht*** mit Blockchiffren daher ***kein*** weiterer Modus mit Blockchiffren..
Eine Triple-DES Entschlüsselung mit 3 verschiedenen Schlüsseln kann mit $\boldsymbol{M = D(D(D(C, K_1), K_2), K_3)}$ beschrieben werden. Dabei ist M der Klartext und C das Chiffrat.	☐ richtig ☑ falsch	Korrekt ist: $\boldsymbol{M = D(E(D(C, K_3), K_2), K_1)}$

Danksagung

- Einige Folien entstammen aus der Vorlesung „Sichere Netzwerkkommunikation“ von Prof. Dr. A. Steffen, Hochschule Rapperswil. An dieser Stelle ein recht herzliches Danke schön an Andreas Steffen.